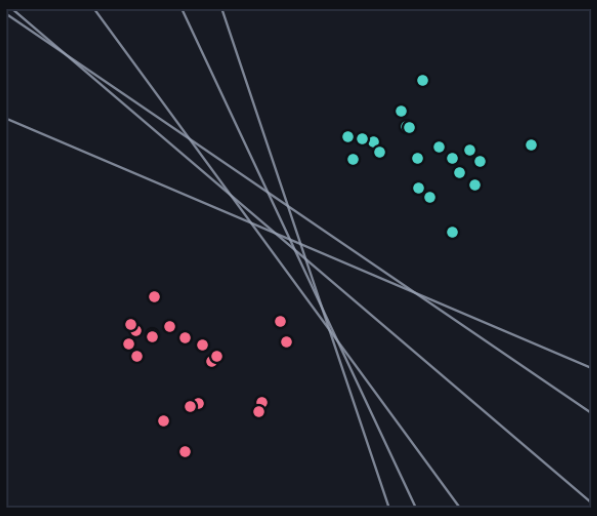
SUPPORT VECTOR MACHINES

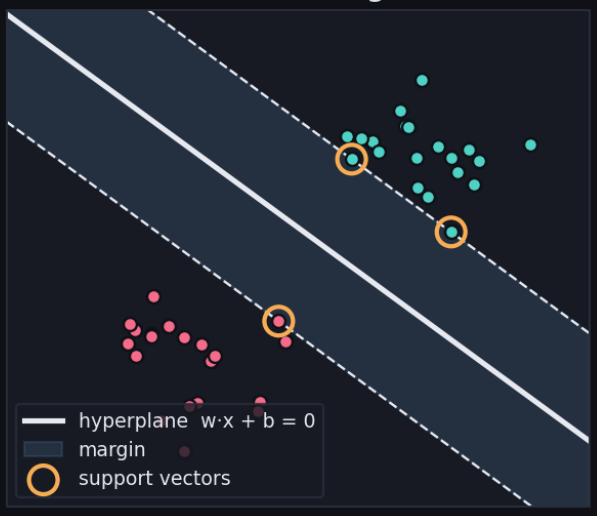
SVM Hyperparameters From Zero to Pro

C · Kernel · Gamma · Degree · Coef0
with intuition, math, visuals, Python code and pro tricks

Many lines separate the classes... which one is best?



SVM picks the line with the WIDEST margin



How to use this tutorial. It moves in one straight line: basic idea → the math behind it → each hyperparameter → tuning → code → professional tricks. Every idea comes with a picture or a worked number example. Coloured boxes mark tips, warnings, math and examples. All figures were generated by real scikit-learn code, so what you see is what the algorithm actually does.

Contents

Contents . . . 2
Part 0 · The Map: What You Will Learn . . . 4
Part 1 · The Basics: The Widest Street . . . 5
1.1 The problem SVM solves . . . 5
1.2 Which line is the best line? . . . 5
1.3 Vocabulary (learn these 5 words) . . . 5
1.4 The math of a hyperplane . . . 6
1.5 When the hard margin breaks . . . 7
Part 2 · Hyperparameter 1: C (Regularization) . . . 8
2.1 Soft margin and slack variables . . . 8
2.2 Low C versus high C . . . 9
2.3 How to tune C . . . 10
Part 3 · Hyperparameter 2: The Kernel . . . 11
3.1 Why straight lines are not enough . . . 11
3.2 The kernel trick (the big idea) . . . 12
3.3 Under the hood: the dual form (pro level) . . . 12
3.4 The four kernels . . . 13
Part 4 · Hyperparameter 3: Gamma . . . 14
4.1 The RBF similarity . . . 14
4.2 Low gamma versus high gamma . . . 14
4.3 Which values to use . . . 15
Part 5 · Hyperparameters 4 and 5: Degree and Coef0 . . . 16
5.1 Degree . . . 16
5.2 Coef0 . . . 16
Part 6 · Putting It Together: C × Gamma and Tuning . . . 18
6.1 The C and gamma interplay . . . 18
6.2 Diagnosing over- and underfitting with a validation curve . . . 18
6.3 Grid search with cross-validation . . . 19
6.4 Coarse-to-fine search . . . 20
6.5 Random search . . . 20
Part 7 · Implementing SVM Hyperparameters in Python . . . 21
7.1 C (regularization parameter) . . . 21
7.2 Kernel type . . . 21
7.3 Gamma . . . 21
7.4 Degree . . . 21

7.5 Coef0 . . . 22
7.6 Automated tuning of all parameters with GridSearchCV . . . 22
7.7 The smarter grid (pro version) . . . 22
7.8 Random search variant . . . 23
Part 8 · Best Practices . . . 24
8.1 Feature scaling is non-negotiable . . . 24
8.2 Coarse-to-fine grid search . . . 24
8.3 Bypass standard SVC for massive datasets . . . 24
8.4 Neutralise class imbalance . . . 25
8.5 Match the kernel to the feature-to-sample ratio . . . 26
8.6 Enable probability estimates only when required . . . 26
Part 9 · Pro Tips (Community Wisdom) . . . 28
9.1 The support-vector 'health check' . . . 28
9.2 The memory-cache hack for speed . . . 28
9.3 Kernel approximation for big data (Nystroem / RBFSampler) . . . 28
9.4 The L1 feature-selection pipeline . . . 29
9.5 Beware the multiclass explosion (OvO vs OvR) . . . 29
9.6 Feature scaling with RobustScaler . . . 30
Part 10 · Cheat Sheet, Troubleshooting and Practice . . . 32
10.1 Complete end-to-end example . . . 32
10.2 Master cheat sheet . . . 33
10.3 Troubleshooting table . . . 33
10.4 Golden rules checklist . . . 33
10.5 Glossary . . . 34
10.6 Practice exercises . . . 34
10.7 Where to go next . . . 34
Part 11 · Train an SVM Like a Pro: Start to Finish . . . 36
11.1 Setup and a first look at the data . . . 36
11.2 Split first, then choose the metric and the CV plan . . . 37
11.3 Baselines: know what you must beat . . . 38
11.4 Build the Pipeline . . . 39
11.5 Stage 1: coarse grid search . . . 39
11.6 Read the results like a pro . . . 40
11.7 Stage 2: fine search and deliberate tie-breaking . . . 40
11.8 The final exam: evaluate on the test set . . . 42
11.9 Diagnose, verify, save and predict . . . 43
11.10 The whole script in one piece . . . 44
11.11 Pro upgrades, checklist and practice . . . 46

Part 0 · The Map: What You Will Learn

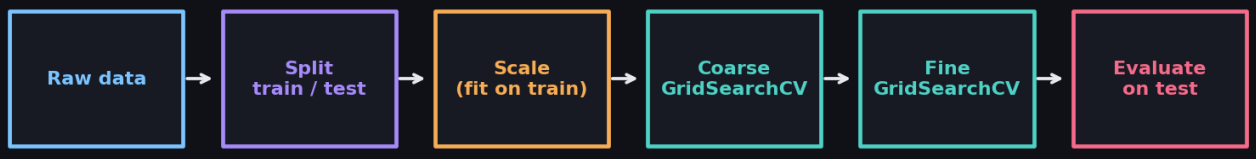
A Support Vector Machine (SVM) is a classifier that draws the **best possible boundary** between classes. *Best* means: the boundary that stays as far away as possible from the training points of both classes. Its behaviour is controlled by only a handful of knobs, called **hyperparameters** (settings you choose *before* training, as opposed to *parameters* such as the weights, which the algorithm learns by itself).

Here are all five knobs in one glance. The rest of the tutorial explains each row in depth.

#	Hyperparameter	What it controls	Applies to	Default (sklearn)	Typical search values
1	**C**	Strictness vs. smoothness (penalty for mistakes)	All kernels	1.0	0.01, 0.1, 1, 10, 100 (log scale)
2	**kernel**	Shape of boundary: straight or curved	—	'rbf'	linear, rbf, poly, sigmoid
3	**gamma**	Reach of a single training point	rbf, poly, sigmoid	'scale'	scale, auto, 0.001 … 10 (log)
4	**degree**	Flexibility of a polynomial curve	poly only	3	2, 3, 4, 5
5	**coef0**	Independent term; mixes low- and high-order terms	poly, sigmoid	0.0	0.0, 0.1, 1

NOTE · Why the order matters

We learn in this order because each idea builds on the previous one: **margin** (Part 1) → **C** (Part 2) → **kernels** (Part 3) → **gamma** (Part 4) → **degree and coef0** (Part 5) → **how they interact and how to tune them** (Part 6) → **code** (Part 7) → **best practices** (Part 8) → **pro tricks** (Part 9) → **cheat sheet, troubleshooting, glossary and exercises** (Part 10).



The professional workflow you will be able to build by the end of this tutorial.

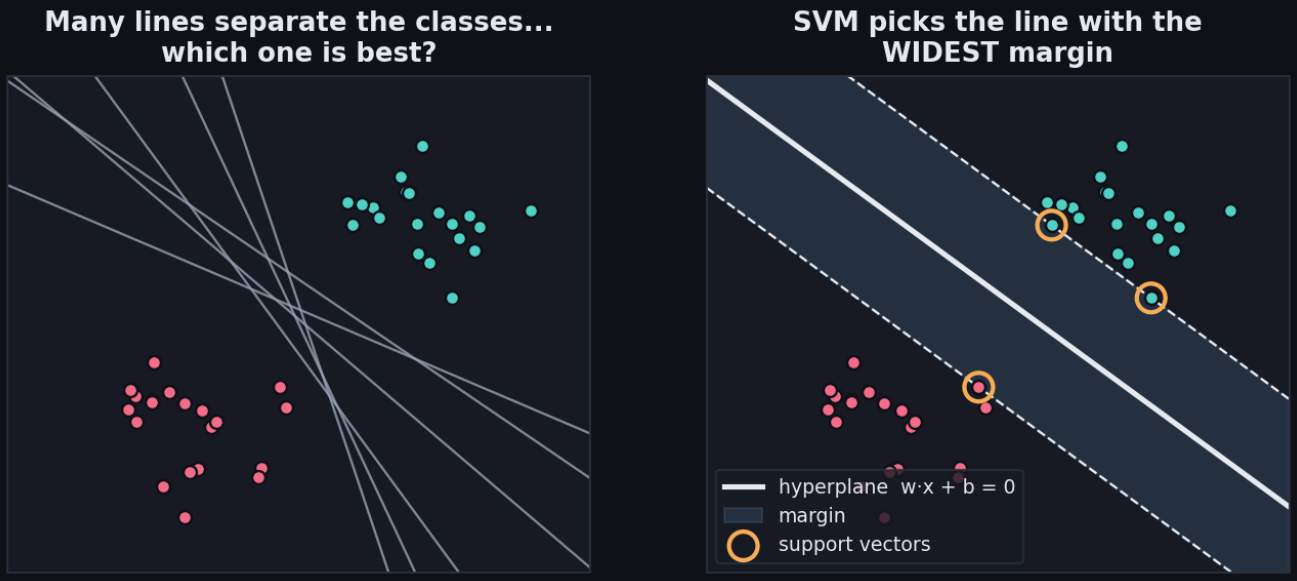
Part 1 · The Basics: The Widest Street

1.1 The problem SVM solves

Imagine you record two numbers for every student: **hours studied** and **hours slept**. Some students passed (teal), some failed (rose). You plot each student as a dot. Now a new student arrives and you must predict pass or fail. The natural approach is to **draw a line that separates the two groups** and see on which side the new dot falls. This task is called **binary classification**.

1.2 Which line is the best line?

Look at the left picture below. Many different lines separate the two groups perfectly. They all make zero mistakes on the training data, but they are not equally good. A line that passes very close to a training point is risky: a new student who is only slightly different could land on the wrong side. **The SVM answers with a simple, powerful rule: choose the line that leaves the largest empty gap to both classes.**



Left: infinitely many separating lines. Right: the SVM line is in the middle of the widest possible empty street.

Think of it as building a road between two towns. You want the road to be as wide as possible without demolishing any house. The road's centre line is the decision boundary; the road's width is the **margin**; the houses touching the road's edge are the **support vectors**.

1.3 Vocabulary (learn these 5 words)

Term	Plain meaning	In the picture
Hyperplane	The decision boundary. A line in 2-D, a flat plane in 3-D, and a 'flat sheet' in higher dimensions.	The solid white line
Margin	The width of the empty street around the hyperplane.	Shaded band between dashed lines
Support vectors	The training points that lie exactly on the edge of the margin (or inside it). They alone define the boundary.	Orange circles
Feature	One measured number per sample (hours studied, sleep, ...).	Axes
Label	The class of a sample. We encode it as y = +1 or y = −1.	Teal = +1, rose = −1

TIP · Key insight

Delete every point that is *not* a support vector and retrain: you obtain exactly the same boundary. This is why SVMs are said to use a **sparse subset** of the data, and why the number of support vectors is a useful health indicator (Part 9).

1.4 The math of a hyperplane

Every point is a vector $\mathbf{x} = (x_1, x_2, \ldots, x_n)$. A hyperplane is described by two things: a weight vector $\mathbf{w}$ (which points perpendicular to the plane) and a bias (offset) $\mathbf{b}$. A point lies exactly on the plane when

$$\mathbf{w} \cdot \mathbf{x} + \mathbf{b} = 0$$

where $\mathbf{w} \cdot \mathbf{x} = w_1x_1 + w_2x_2 + \ldots + w_nx_n$ is the dot product. For a new point the SVM computes the score $f(x) = w \cdot x + b$ and predicts by its **sign**:

$$\text{prediction} = \text{sign}(w \cdot x + b) \rightarrow +1 \text{ if score} > 0, -1 \text{ if score} < 0$$

The two edges of the street are the lines where the score equals exactly +1 and −1:

$$w \cdot x + b = +1 \text{ (edge of class +1)} \quad w \cdot x + b = -1 \text{ (edge of class −1)}$$

Margin width. The distance from any point x to the plane is $|w \cdot x + b| / \|w\|$, where $\|w\| = \sqrt{(w_1^2 + \ldots + w_n^2)}$ is the length of w. For a support vector the score is ±1, so it sits $1/\|w\|$ from the plane. Both sides together give the total street width:

$$\text{margin width} = 2 / \|w\|$$

To make the street *wide* we must make $\|w\|$ *small*. That gives the SVM's optimisation problem, called the **hard-margin SVM**:

$$\text{minimise } \tfrac{1}{2} \|w\|^2 \text{ subject to } y_i (w \cdot x_i + b) \geq 1 \text{ for every training point } i$$

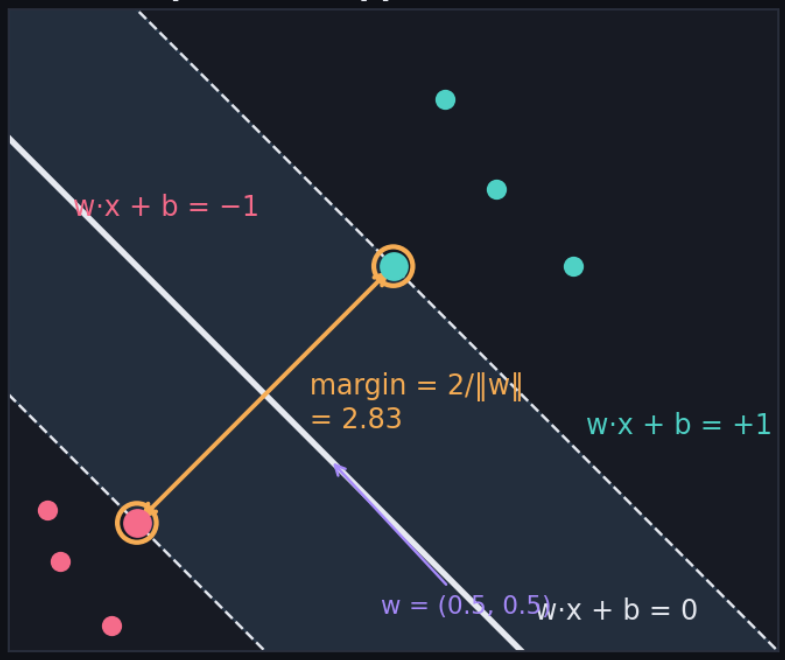
In words: *make the street as wide as possible (small $\|w\|$), while every point stays on the correct side and outside the street* (the constraint says: class +1 points must score ≥ +1, class −1 points must score ≤ −1; multiplying by $y_i = \pm 1$ merges both cases into one inequality). The ½ and the square are only there to make the calculus neat; minimising $\frac{1}{2}\|w\|^2$ and minimising $\|w\|$ give the same answer.

EXAMPLE · Worked example with real numbers

Class −1 has a point at (1, 1). Class +1 has a point at (3, 3). These two are the support vectors. Try **w = (0.5, 0.5)** and **b = −2**.

- Score of (3,3): 0.5·3 + 0.5·3 − 2 = **+1** ✓ (on the +1 edge).
- Score of (1,1): 0.5·1 + 0.5·1 − 2 = **−1** ✓ (on the −1 edge).
- $\|w\| = \sqrt{(0.25 + 0.25)} = 0.707$, so margin = 2 / 0.707 = **2.83**, which is exactly the straight-line distance between (1,1) and (3,3) = $\sqrt{(2^2 + 2^2)} = 2.83$ ✓.
- New point (4, 2): score = 2 + 1 − 2 = +1 → predicted class +1.

Worked example: two support vectors (1,1) and (3,3)



Geometry of the worked example. The dashed lines are the ±1 edges; the margin is the arrow.

1.5 When the hard margin breaks

The hard-margin SVM has two serious limitations: (1) it only works if the classes can be separated perfectly by a straight line; (2) even one noisy outlier can force a terrible, very narrow street. Real data is messy, so we need a forgiving version. That forgiveness is controlled by the first and most important hyperparameter: **C**.

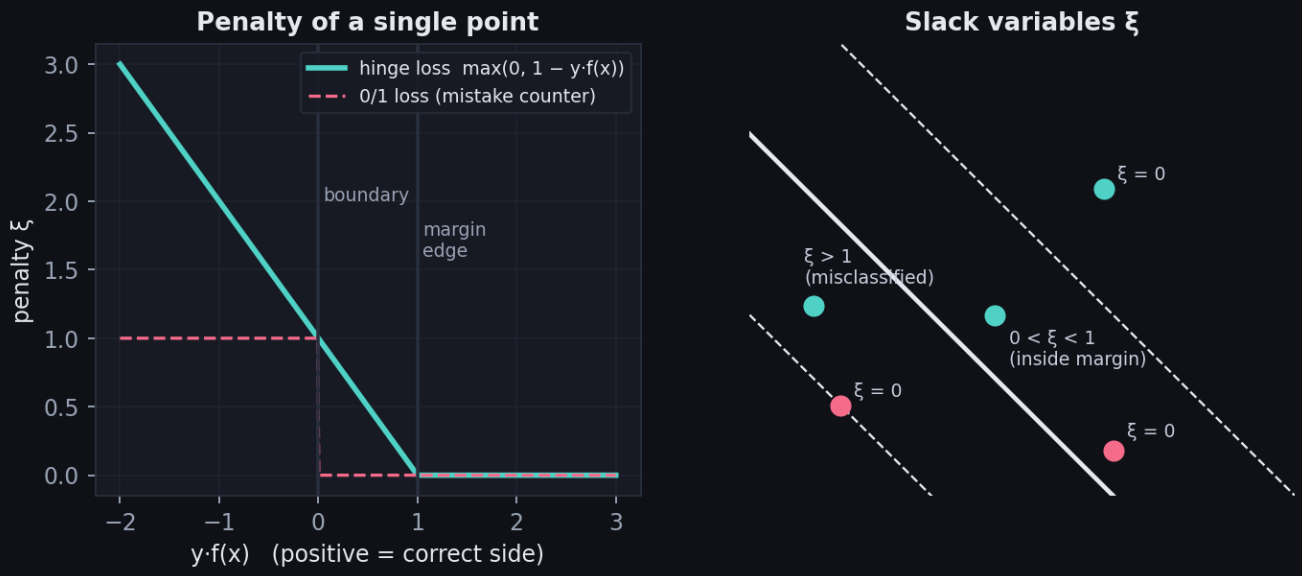
Part 2 · Hyperparameter 1: C (Regularization)

C is the most frequently tuned hyperparameter of every SVM implementation. It sets the trade-off between two conflicting goals: **(a)** a wide margin (simple, smooth boundary that generalises) and **(b)** few misclassified training points (low training error).

2.1 Soft margin and slack variables

The soft-margin SVM allows points to break the rules, but every violation is *charged*. For each training point i we introduce a **slack variable ξ_i** (the Greek letter 'xi') measuring how badly it violates the margin:

- **$\xi = 0$**: the point is correctly classified and outside (or on the edge of) the street. No penalty.
- **$0 < \xi < 1$**: the point is on the correct side but has crept *inside* the street.
- **$\xi > 1$**: the point is on the *wrong side* of the boundary, i.e. misclassified.



Left: the hinge loss is the penalty ξ as a function of the point's score. Right: slack values in a picture.

The slack of a point is exactly its **hinge loss**: $\xi_i = \max(0, 1 - y_i \cdot f(x_i))$. The soft-margin objective from your notes is:

$$\text{minimise } \tfrac{1}{2}\|w\|^2 + C \cdot \sum \xi_i \text{ (sum over all n training points, i = 1 … n)}$$

- **$\frac{1}{2}\|w\|^2$** rewards a *wide* margin (simplicity).
- **$C \cdot \sum \xi_i$** punishes rule-breaking (errors). **C is the price per unit of slack.**

WATCH OUT · C is the INVERSE of regularization strength

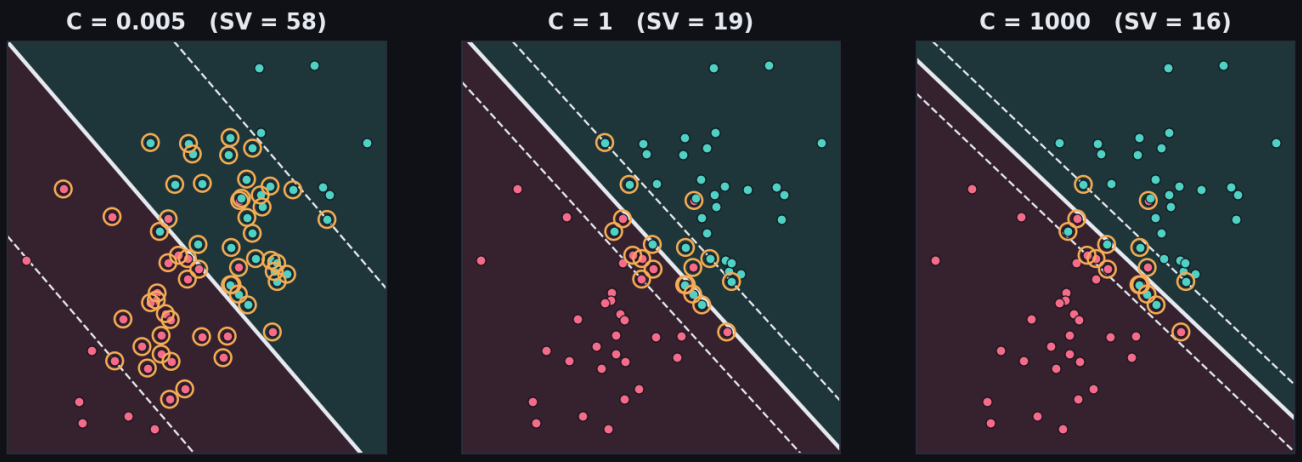
The name 'regularization parameter' confuses many beginners. Regularization means 'keep the model simple'. In SVM, **small C = strong regularization** (simple, smooth, wide margin) and **large C = weak regularization** (complex, strict). Compare with Ridge/Lasso regression, where a large alpha means strong regularization: the direction is reversed. Rule of thumb: $C \approx 1 / \lambda$.

EXAMPLE · Worked example: how C changes the winner

Two candidate boundaries for the same data:
• **A (wide street)**: $\|w\|^2 = 1$ but it lets several points slip, total slack $\sum\xi = 10$.
• **B (narrow street)**: $\|w\|^2 = 25$ but nearly all points are respected, $\sum\xi = 1$.

Cost = $\frac{1}{2}\|w\|^2 + C\cdot\sum\xi$:
• C = 0.01 → A: 0.5 + 0.1 = **0.6**; B: 12.5 + 0.01 = 12.51 → **A wins** (wide).
• C = 1 → A: 0.5 + 10 = **10.5**; B: 12.5 + 1 = 13.5 → **A wins**.
• C = 100 → A: 0.5 + 1000 = 1000.5; B: 12.5 + 100 = **112.5** → **B wins** (narrow, strict).
The two costs are equal when 0.5 + 10C = 12.5 + C, i.e. at C = 12/9 ≈ 1.33. Below it the SVM prefers the wide street; above it, the narrow one.

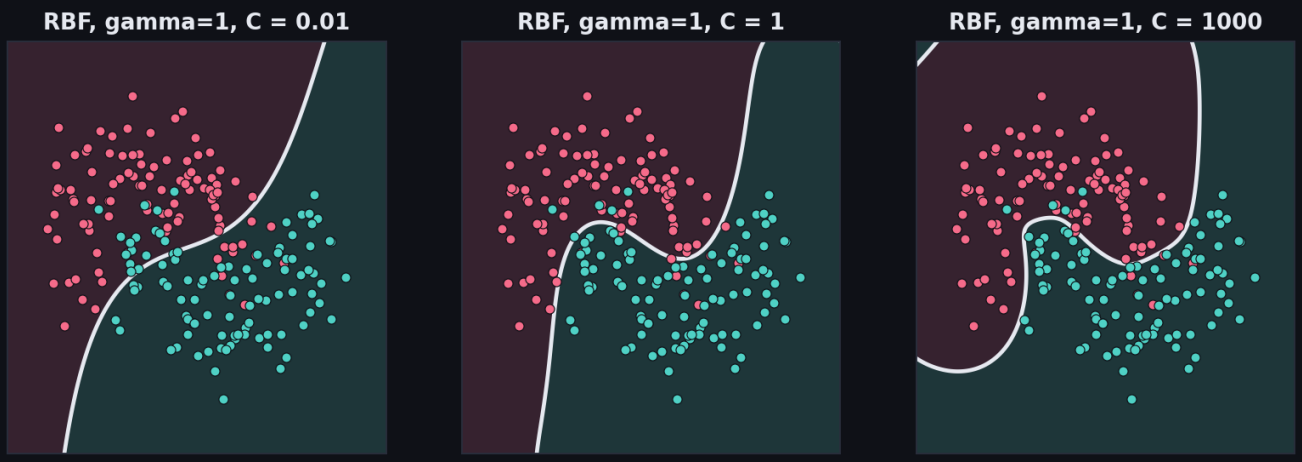
2.2 Low C versus high C



Same data, linear kernel. Dashed lines = margin, orange rings = support vectors. As C grows the street narrows and fewer points remain support vectors.

	Low C (0.001 · 0.01 · 0.1)	High C (10 · 100 · 1000)
Behaviour	Prioritises a large, smooth margin, even if several training points are misclassified. Treats stray points as noise, not rules.	Heavily penalises any misclassification. Prioritises classifying every training point correctly over margin width.
Visual result	Wide margin, smooth and flatter boundary.	Very narrow margin, complex contorted boundary that weaves tightly around the training data.
Risk	High **bias** → underfitting. If C is too low the model ignores the patterns and fails to separate the classes.	High **variance** → severe overfitting. Hypersensitive to noise and outliers, so poor performance on unseen data.
Support vectors	Many (lots of points inside the wide street).	Few (only points right at the narrow street).
Fix	Underfitting → increase C.	Overfitting → decrease C.

With a non-linear kernel the effect is even more dramatic: a huge C forces the curve to twist around individual noisy points.



RBF kernel on noisy 'two moons'. C = 0.01 is too stiff (underfit), C = 1 is balanced, C = 1000 chases noise (overfit).

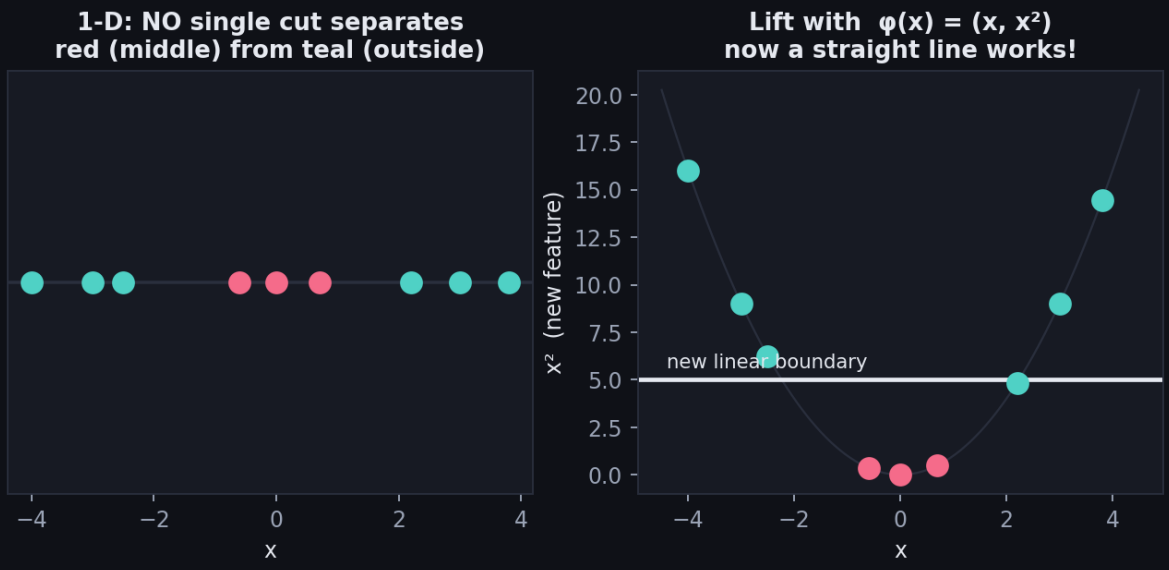
2.3 How to tune C

- **Always on a logarithmic scale**: 10^{-3} … 10^{3}, i.e. 0.01, 0.1, 1, 10, 100. Going from C = 1 to C = 2 barely changes anything; going from 1 to 10 does.
- **Feature scaling is mandatory first.** The best C depends on the scale of your features. Standardise to mean 0 and variance 1 (Part 8).
- C is used in **every** SVM model, whatever the kernel. It is the first knob you should touch.
- Overfitting → *decrease* C (smoother boundary). Underfitting → *increase* C (capture data more strictly).

Part 3 · Hyperparameter 2: The Kernel

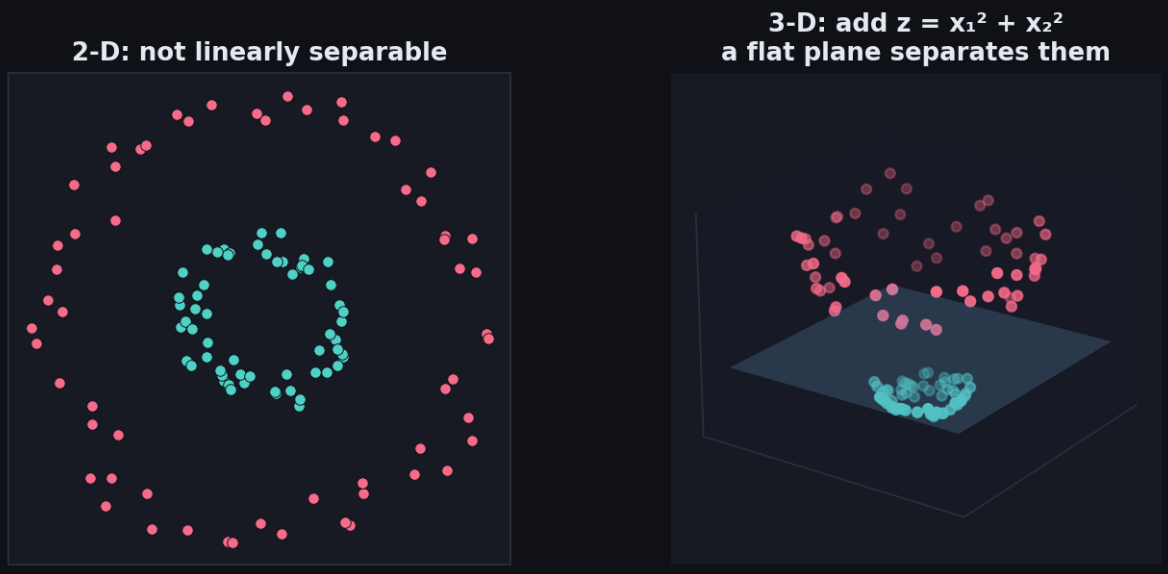
3.1 Why straight lines are not enough

Some data cannot be separated by any straight line. Take the 1-D example below: red points sit in the middle, teal points on both sides. No single cut on the number line works. But if we add a **new feature**, x^2, each point x becomes a 2-D point (x, x^2), and suddenly a straight horizontal line separates the classes perfectly.



Lifting 1-D data to 2-D with the feature map $\varphi(x) = (x, x^2)$.

The same trick works in 2-D. Concentric circles cannot be split by a line, but adding a third coordinate $z = x_1^2 + x_2^2$ (the squared distance from the centre) lifts the inner circle low and the outer circle high, and a flat plane cuts between them.



Lifting 2-D circles into 3-D. A flat plane in 3-D becomes a circle when projected back to 2-D.

The **kernel function** is what lets the SVM draw such a flat hyperplane in a higher dimension so that, in the original space, it looks like a curve.

3.2 The kernel trick (the big idea)

Mapping every point explicitly to a higher-dimensional space is expensive, and sometimes the target space is infinite-dimensional. The SVM has a remarkable property: it only ever needs **dot products between pairs of points**, never the points themselves. A kernel computes the dot product in the high-dimensional space *directly from the original vectors*, without ever building the high-dimensional vectors:

$$K(x_i, x_j) = \varphi(x_i) \cdot \varphi(x_j)$$

EXAMPLE · Verify the trick with numbers

Take 2-D points x = (1, 2) and z = (3, 1) and the kernel $K(x, z) = (x \cdot z)^2$.
Shortcut (kernel): $x \cdot z = 3 + 2 = 5$, so $K = 5^2 = \mathbf{25}$.
Long way (explicit map): use $\varphi(x) = (x_1^2, \sqrt{2} \cdot x_1 x_2, x_2^2)$. Then φ(x) = (1, 2.83, 4) and φ(z) = (9, 4.24, 1). Dot product = 9 + 12 + 4 = **25** ✓.
Same answer, but the kernel never left 2-D. For an RBF kernel the long way would need infinitely many coordinates, yet the shortcut is one line.

3.3 Under the hood: the dual form (pro level)

Using Lagrange multipliers α_i (one per training point), the SVM problem can be rewritten in its **dual form**, where the data appears only through K:

$$\text{maximise } \sum \alpha_i - \tfrac{1}{2} \sum\sum \alpha_i \alpha_j y_i y_j K(x_i, x_j)$$
$$\text{subject to } 0 \le \alpha_i \le C \text{ and } \sum \alpha_i y_i = 0$$

and predictions for a new point x use only the support vectors (the points with $\alpha_i > 0$):

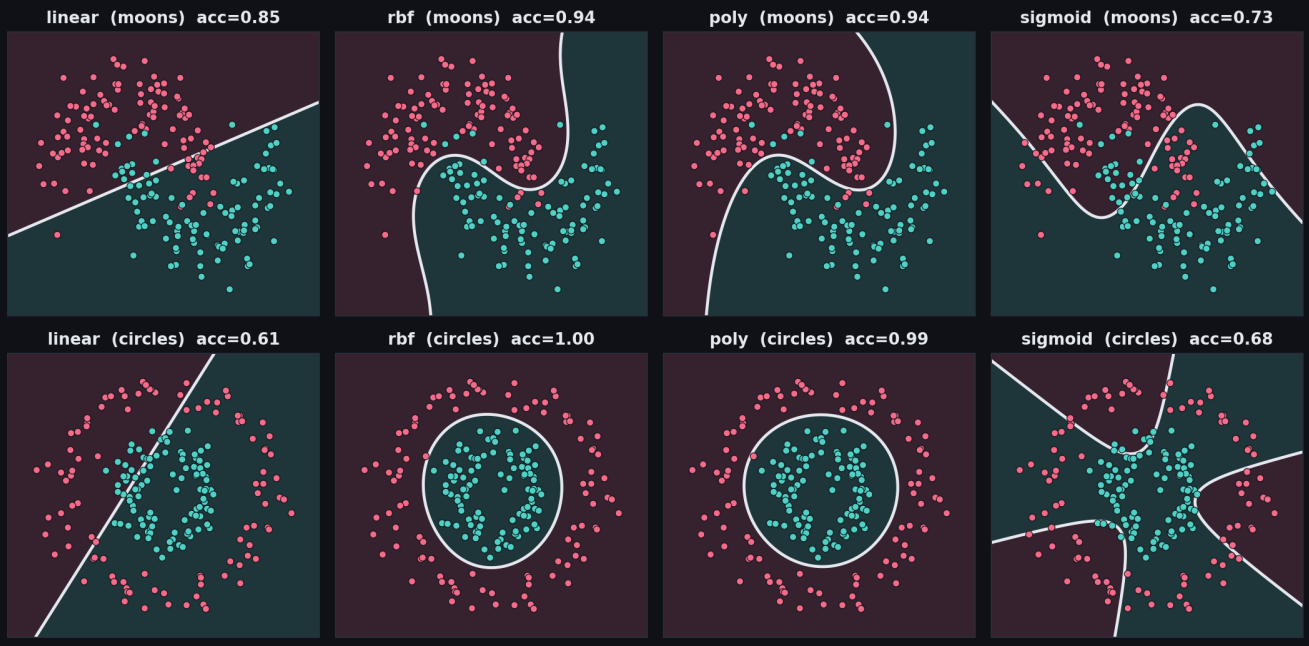
$$f(x) = \text{sign}\left(\sum_{\text{support vectors}} \alpha_i y_i K(x_i, x) + b \right)$$

NOTE · Where C hides in the dual

C is the **upper cap** on every α_i. Points with α = 0 are irrelevant (not support vectors). Points with 0 < α < C sit exactly on the margin edge. Points with α = C are the margin violators (inside the street or misclassified). A tiny C caps every α at a tiny number, so no single point can have much influence: smooth boundary. A huge C lets single points pull hard: contorted boundary. This is the same story as in Part 2, seen from the other side.

3.4 The four kernels

Kernel	Formula $K(x_i, x_j)$	Extra hyperparameters	Use when
linear	$x_i \cdot x_j$	none (only C)	Linearly separable data; text classification, sentiment analysis, genomics; features far outnumber samples; massive datasets.
rbf	$\exp(-\gamma \lVert x_i - x_j \rVert^2)$	gamma	**Default starting point** for non-linear data. Maps to an infinite-dimensional space; adapts to almost any boundary.
poly	$(\gamma (x_i \cdot x_j) + r)^d$	degree d, gamma, coef0 r	Image processing or tasks where feature interactions are inherently polynomial. Computationally heavier.
sigmoid	$\tanh(\gamma (x_i \cdot x_j) + r)$	gamma, coef0 r	Proxy for neural-network behaviour (a two-layer perceptron). Rarely the best choice in practice.



All four kernels (C = 1) on two datasets. Linear fails on curved data; RBF and poly handle both; sigmoid is unpredictable.

3.5 Advantages and disadvantages

- **Linear**: extremely fast to train, no kernel-specific hyperparameters (only C), so optimisation is rapid. Easy to interpret via the weights w.
- **RBF**: highly flexible, works well with few assumptions. Needs both C and gamma tuned together.
- **Polynomial**: computationally intensive and requires tuning **three** separate hyperparameters (C, degree d, and coef0 r). Very high degrees overfit.
- **Sigmoid**: only behaves like a valid kernel for some parameter ranges (it is not always positive semi-definite), so it can give erratic results. Start with rbf; try sigmoid only if you have a reason.

> **TIP · The safe default recipe**
>
> Not sure what to use? Try **linear** first (fast baseline). If it underfits, switch to **rbf** and tune C and gamma. Only try poly if you have domain knowledge that polynomial interactions matter.

Part 4 · Hyperparameter 3: Gamma

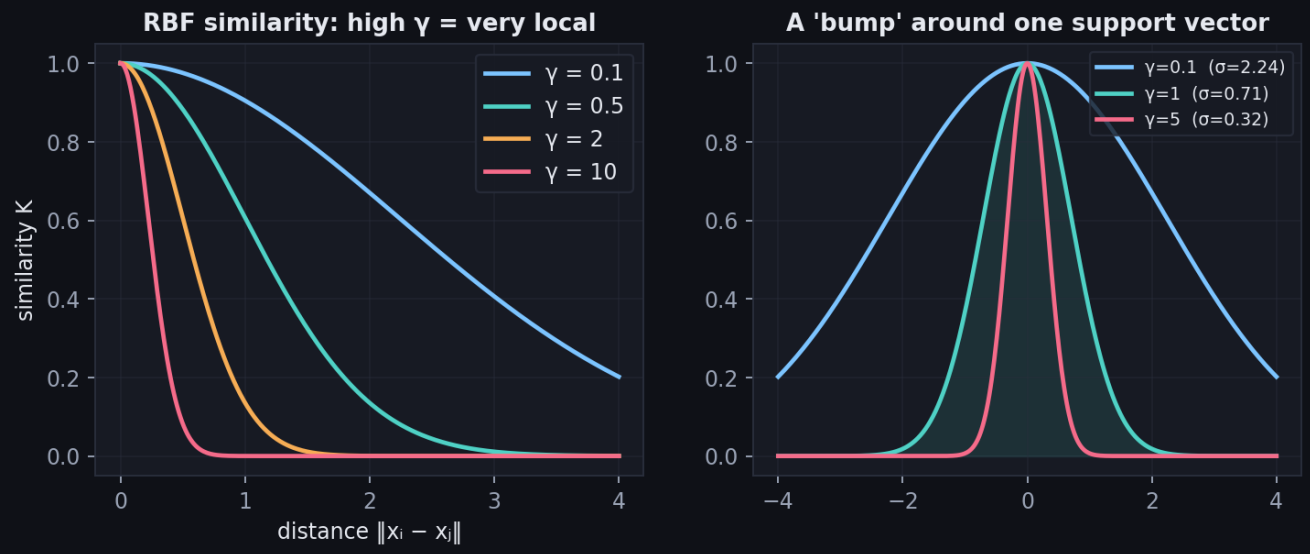
When using the RBF kernel (the most common choice for non-linear data), gamma is the second most critical hyperparameter, tuned together with C. It defines the **radius of influence of a single training example**, specifically of the support vectors.

4.1 The RBF similarity

The RBF kernel is a similarity score between two points, between 0 (very different) and 1 (identical):

$$K(x_i, x_j) = \exp(-\gamma \cdot \|x_i - x_j\|^2) \text{ with } \gamma = 1 / (2\sigma^2)$$

$\|x_i - x_j\|^2$ is the squared Euclidean distance. Gamma multiplies it, so **gamma controls how fast similarity dies as points move apart**. It is the inverse of the radius of influence: σ is the 'width' of the bell curve around each support vector.

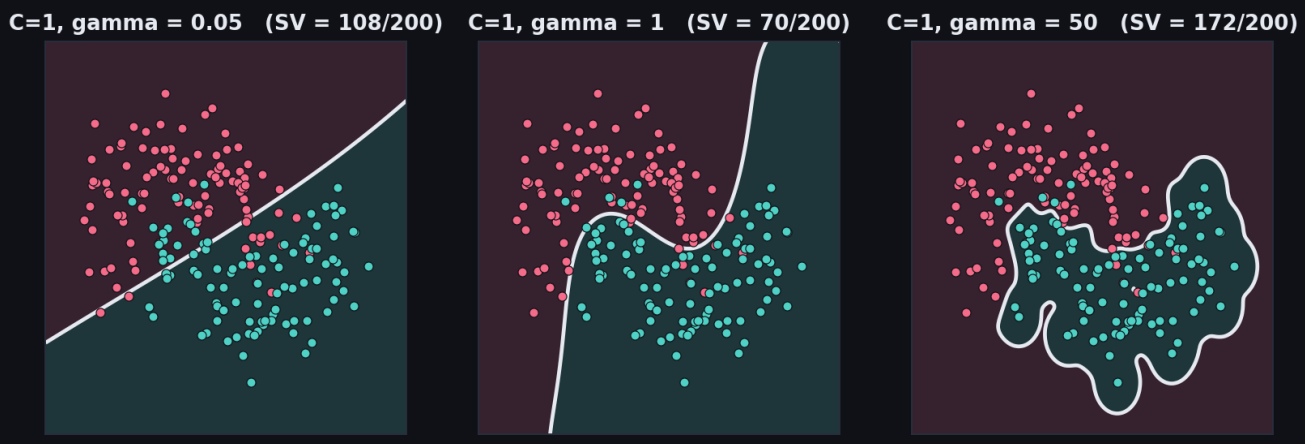


Left: similarity versus distance for different gamma. Right: the Gaussian 'bump' each support vector contributes.

Distance between points	γ = 0.1	γ = 1	γ = 10
0.5	exp(−0.025) = 0.975	exp(−0.25) = 0.779	exp(−2.5) = 0.082
1	exp(−0.1) = 0.905	exp(−1) = 0.368	exp(−10) = 0.000045
2	exp(−0.4) = 0.670	exp(−4) = 0.018	exp(−40) ≈ 0

At γ = 10, two points a distance of 1 apart are already 'strangers' (similarity 0.00005). At γ = 0.1 they are still 90% similar. The final SVM decision is a weighted sum of these bumps, one per support vector: many narrow bumps produce a jagged surface; few wide bumps produce a smooth one.

4.2 Low gamma versus high gamma



RBF SVM, C = 1. Left: gamma too low (almost a straight line). Middle: balanced. Right: gamma too high (islands around each point).

	Low gamma (0.0001 · 0.001 · 0.01)	High gamma (1 · 10 · 100)
Radius of influence	Very large: a single point influences a massive area of feature space.	Extremely small: a point only affects the boundary right next to it.
Visual result	Incredibly smooth, almost linear boundary; the model considers the overall class distribution rather than local variations.	Boundary behaves like 'islands' tightly drawn around individual data points.
Risk	**Underfitting**: loses non-linear nuances, acts almost like a linear SVM.	**Catastrophic overfitting**: memorises the training set, cannot generalise.
Fix	Increase gamma to capture complex, intricate patterns.	Decrease gamma to cure overfitting.

4.3 Which values to use

- **'scale'** (default, recommended): gamma = 1 / (n_features × X.var()). It adapts to the variance of your data, so it works sensibly even when features have different overall magnitudes.
- **'auto'**: gamma = 1 / n_features. Ignores the data's variance, so it is only sensible when features are already standardised.
- **Explicit floats on a log scale**: 0.001, 0.01, 0.1, 1, 10.
- **Only applicable to rbf, poly and sigmoid.** The linear kernel ignores gamma completely.

MATH · Quick sanity computation

Suppose you have 4 standardised features (variance ≈ 1 overall). Then gamma('scale') = 1 / (4 × 1) = 0.25 and gamma('auto') = 1 / 4 = 0.25, identical. If your data variance were 9 (unscaled), 'scale' would give 1 / (4 × 9) = 0.028 while 'auto' would still give 0.25: 'scale' automatically compensates.

WATCH OUT · Gamma and C are coupled

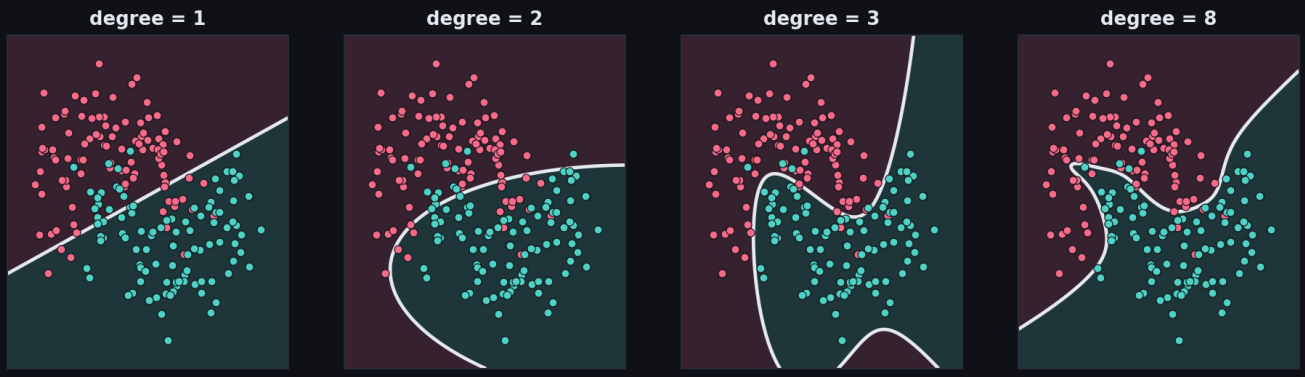
A good gamma depends on C and vice versa. Never tune one in isolation. Part 6 shows how to search both together.

Part 5 · Hyperparameters 4 and 5: Degree and Coef0

These two only matter for the **polynomial** kernel (coef0 also for sigmoid). If you use linear or rbf, they are ignored completely.

5.1 Degree

The polynomial kernel is $K(x, z) = (\gamma\ (x \cdot z) + r)^d$. The exponent **d** is the **degree**: it sets the flexibility of the decision boundary. Degree 1 is a straight line. Degree 2 can bend once (parabola/ellipse). Higher degrees create more wiggles.



Polynomial kernel with increasing degree. Degree 1 underfits; degree 8 twists and wiggles (overfitting risk).

- Preferred values: **2, 3, 4, 5**. Default is 3.
- Used **only** when kernel='poly'; ignored by every other kernel.
- Increasing the degree captures more complex curves, but excessively high degrees cause severe overfitting *and* heavy computation.

MATH · Why high degrees are expensive

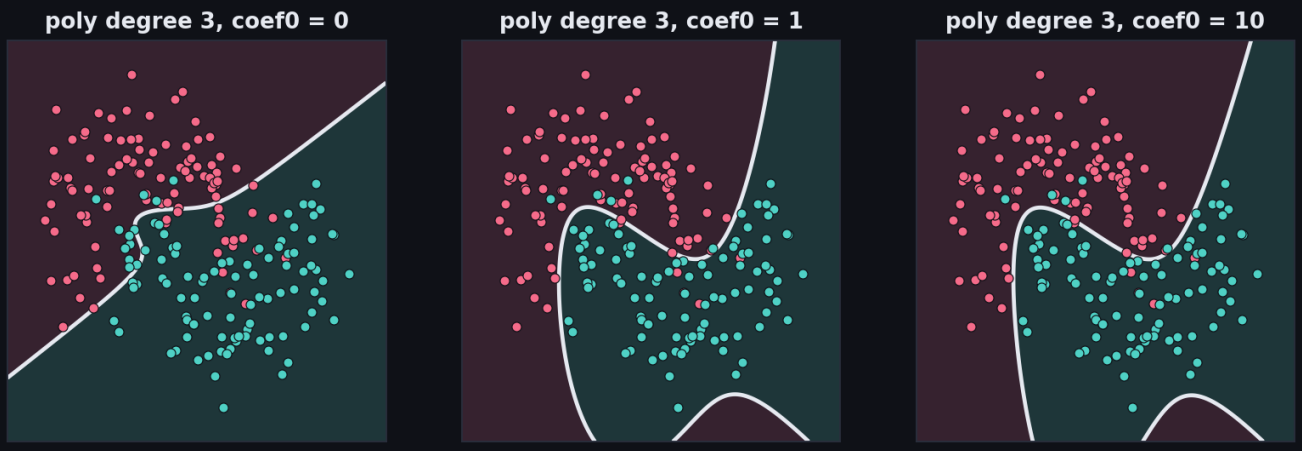
With n = 100 input features, a degree-3 polynomial kernel implicitly works in a space with about C(102, 3) = 171,700 dimensions of pure degree-3 terms (and 176,851 when lower-order terms are included). The kernel trick avoids building them, but the numbers inside the kernel can become huge or tiny (values raised to the power d), which hurts numerical stability and speed. Keep d small (2 or 3) and scale your features.

5.2 Coef0

Coef0 is the independent term **r** in $(\gamma\ x \cdot z + \mathbf{r})^d$ (and in $\tanh(\gamma\ x \cdot z + \mathbf{r})$ for sigmoid). It controls how much the model is influenced by **high-degree versus low-degree** terms.

EXAMPLE · See it with algebra

Take one feature and d = 2:
• r = 0: $(x)^2 = \mathbf{x^2}$. Only the pure degree-2 term exists.
• r = 1: $(x + 1)^2 = \mathbf{x^2 + 2x + 1}$. Now degree-2, degree-1 and constant terms are all present.
• r = 10: $(x + 10)^2 = x^2 + 20x + 100$. The constant and linear terms dominate over x^2, so the kernel behaves more like a simpler, lower-degree model.
So **larger r shifts weight towards lower-order terms**, smoothing the influence of the highest-degree term.



Polynomial degree 3 with coef0 = 0, 1, 10. Same degree, but the mix of low- and high-order terms changes the shape.

- Preferred values: usually left at the default **0.0**; tune with small increments such as **0.1 or 1**.
- Only significant for **poly** and **sigmoid** kernels. Adjusting it balances the bias of the kernel against the influence of higher-order features.

TIP · Practical advice

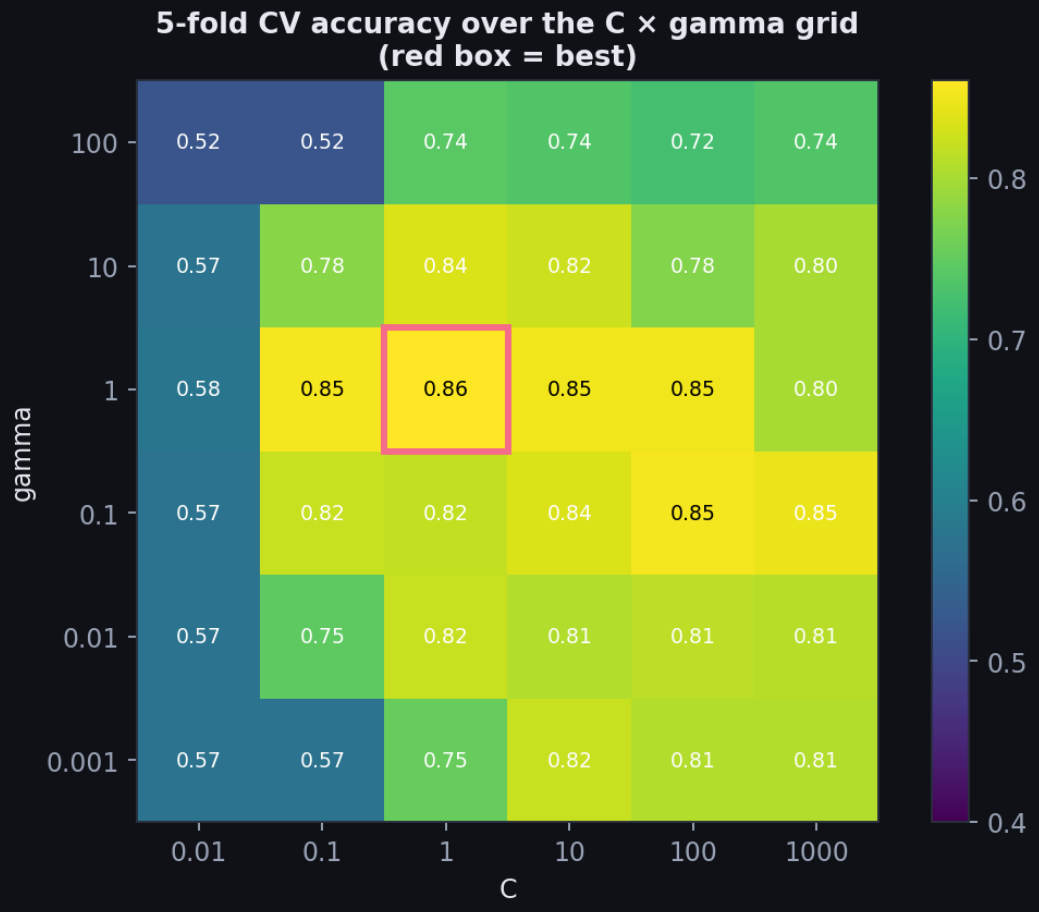
For poly, coef0 = 1 is a very common and useful starting choice: it makes the kernel include all lower-order terms, so it can represent the full polynomial rather than only the pure degree-d part.

Part 6 · Putting It Together: C × Gamma and Tuning

6.1 The C and gamma interplay

C and gamma cannot be tuned in isolation; they are deeply coupled.

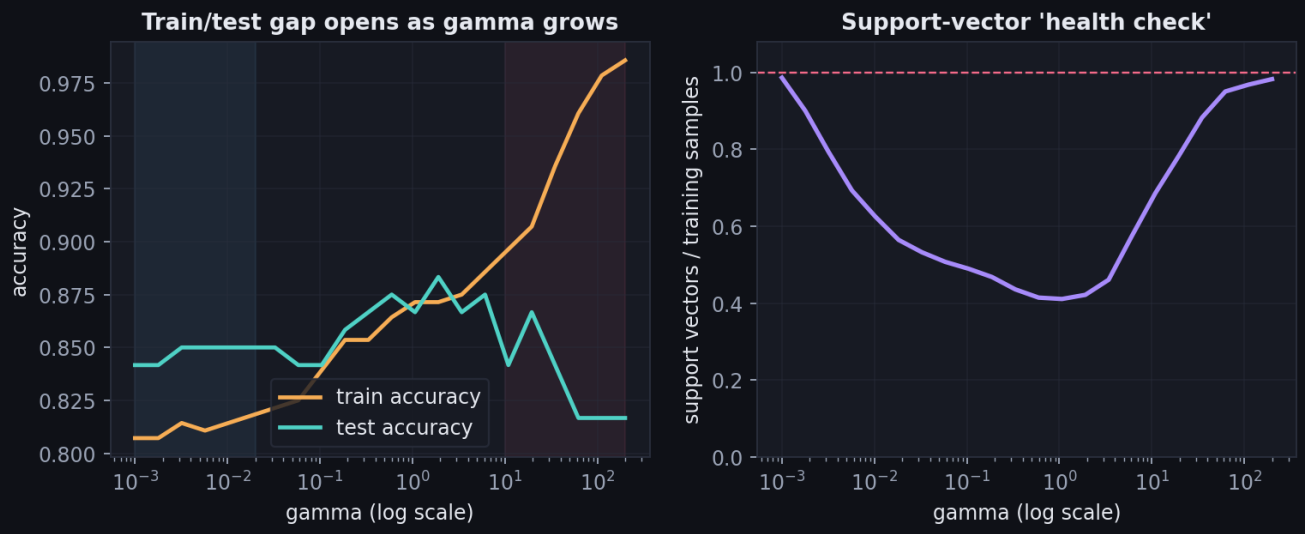
- If **gamma is too high**, the model overfits regardless of what C is set to.
- If **gamma is low**, the model's complexity is driven almost entirely by C.
- Good models often lie along a **diagonal ridge**: large gamma paired with small C, or small gamma paired with large C. Both settings can produce a similar complexity.



Real 5-fold cross-validated accuracy for every (C, gamma) pair on a noisy two-moons dataset. Bright cells = good. The red box is the best.

How to read the heat-map: the top-left/bottom-right corners are poor (too simple or too wiggly). Notice how the good region forms a band, not a single point. That is why a joint search is essential.

6.2 Diagnosing over- and underfitting with a validation curve



Left: as gamma increases, training accuracy climbs towards 100% while test accuracy falls (the widening gap = overfitting). Right: the fraction of training points that are support vectors.

Symptom	Diagnosis	Action
Low train accuracy AND low test accuracy	Underfitting (too simple)	Increase C, increase gamma, try a more flexible kernel, add features
High train accuracy, much lower test accuracy	Overfitting (too complex)	Decrease C, decrease gamma, lower degree, get more data, remove noisy features
Train ≈ test, both high	Good fit	Stop tuning; evaluate once on the held-out test set

6.3 Grid search with cross-validation

Guessing manually is highly inefficient. The most effective approach is a **grid search combined with cross-validation**:

- **Grid search** tries every combination in a table of candidate values, such as C ∈ {0.1, 1, 10, 100} and gamma ∈ {1, 0.1, 0.01, 0.001} (16 combinations).
- **k-fold cross-validation** splits the training data into k equal parts (folds). For each combination, it trains k times: each time using k−1 folds for training and the remaining fold for validation. The k validation scores are averaged. With cv=5 and 16 combinations, that is 80 model fits.
- The combination with the highest average validation score wins. The test set is **never** touched during this process, so it remains an honest final exam.

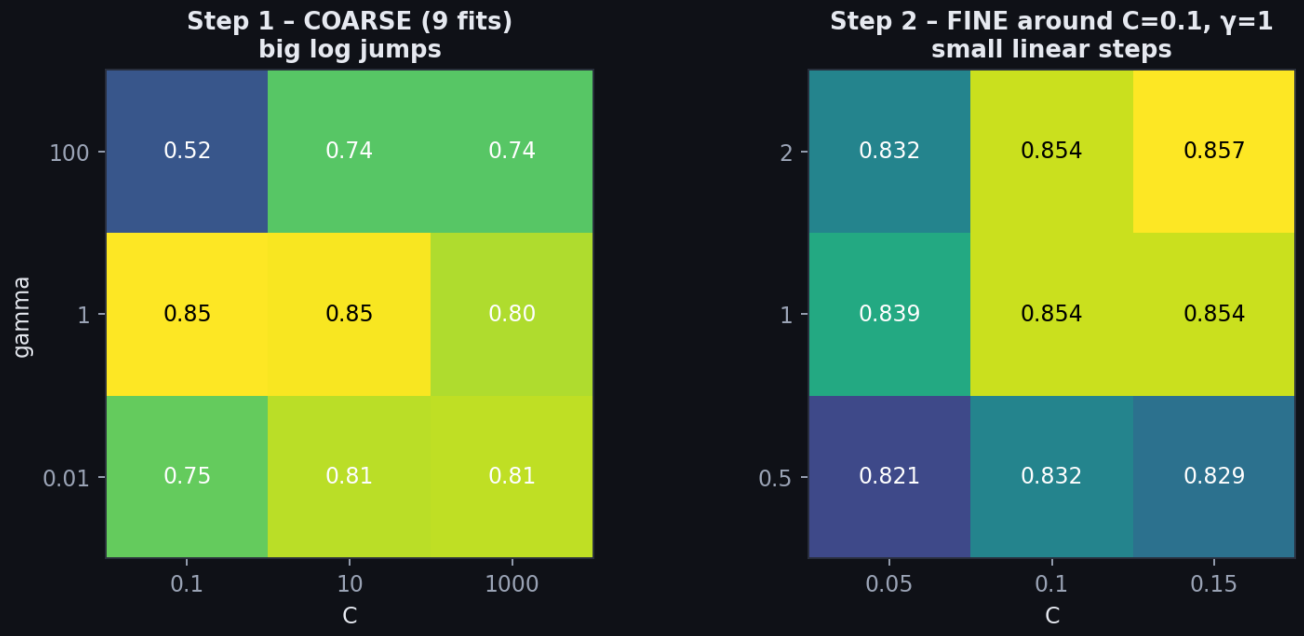
MATH · Cost of a grid

Number of fits = (number of combinations) × k. Your notes' full grid (4 C × 3 kernels × 4 gamma × 2 degree × 2 coef0) is 192 combinations × 5 folds = **960 fits**, and many are wasteful, because degree and coef0 do nothing for 'rbf' and 'linear', and gamma does nothing for 'linear'. See the smarter grid in Part 7.

6.4 Coarse-to-fine search

A giant, granular grid takes enormous time. The professional approach has two steps:

- **Coarse search**: wide range, big logarithmic jumps, e.g. C: [0.1, 10, 1000], gamma: [0.01, 1, 100] (only 9 combinations).
- **Fine search**: once you find the promising region (e.g. C = 10, gamma = 1), run a second grid with small *linear* steps clustered around those winners, e.g. C: [5, 10, 15], gamma: [0.5, 1, 2].



Left: a 9-cell coarse grid locates the region. Right: a 9-cell fine grid zooms into it. 18 fits instead of hundreds.

6.5 Random search

RandomizedSearchCV samples random combinations from distributions (e.g. log-uniform for C and gamma). With the same budget it often finds a better optimum than a coarse grid, because it tries many *distinct* values of each hyperparameter instead of repeating the same few. It is the go-to method when you have more than 2–3 hyperparameters.

Part 7 · Implementing SVM Hyperparameters in Python

SVMs in scikit-learn are implemented by the **SVC** (Support Vector Classification) class. Before setting any parameter, import the model:

```
from sklearn.svm import SVC
```

7.1 C (regularization parameter)

C is passed as a float or integer. The default is 1.0.

```
# High C: strict margin, prone to overfitting
svm_high_c = SVC(C=100.0)

# Low C: wide margin, prone to underfitting
svm_low_c = SVC(C=0.01)

# Fit the model to your scaled training data
svm_high_c.fit(X_train_scaled, y_train)
```

7.2 Kernel type

The kernel parameter takes a string that specifies the mathematical transformation. The default is 'rbf'.

```
# Linear kernel for linearly separable or high-dimensional data
svm_linear = SVC(kernel='linear')

# RBF kernel for non-linear data (default)
svm_rbf = SVC(kernel='rbf')

# Polynomial kernel
svm_poly = SVC(kernel='poly')

# Sigmoid kernel
svm_sigmoid = SVC(kernel='sigmoid')
```

7.3 Gamma

Used when kernel is 'rbf', 'poly' or 'sigmoid'. It accepts a float, or the strings 'scale' (default) or 'auto'.

- **'scale'** uses 1 / (n_features * X.var()).
- **'auto'** uses 1 / n_features.

```
# High gamma: tight decision boundaries around data points
svm_high_gamma = SVC(kernel='rbf', gamma=10.0)

# Low gamma: broad, smooth decision boundaries
svm_low_gamma = SVC(kernel='rbf', gamma=0.001)

# Using the default variance-based scaling
svm_scale_gamma = SVC(kernel='rbf', gamma='scale')
```

7.4 Degree

An integer used exclusively when kernel='poly'. It is ignored by all other kernels. The default is 3.

```
# 2nd degree polynomial
svm_poly_2 = SVC(kernel='poly', degree=2)

# 5th degree polynomial (computationally expensive)
svm_poly_5 = SVC(kernel='poly', degree=5)
```

7.5 Coef0

A float representing the independent term in the kernel function, used only by 'poly' and 'sigmoid'. The default is 0.0.

```
# Adjusting the independent term to balance high/low order features
svm_poly_coef = SVC(kernel='poly', degree=3, coef0=1.5)
```

7.6 Automated tuning of all parameters with GridSearchCV

Setting values one by one is inefficient; professionals use GridSearchCV to test combinations automatically. Here is the version from your notes:

```
from sklearn.model_selection import GridSearchCV
from sklearn.svm import SVC

# 1. Define the parameter grid with the values you want to test
param_grid = {
    'C': [0.1, 1, 10, 100],
    'kernel': ['rbf', 'linear', 'poly'],
    'gamma': ['scale', 0.1, 0.01, 0.001],
    'degree': [2, 3],       # Only affects 'poly'
    'coef0': [0.0, 1.0]     # Only affects 'poly'
}

# 2. Initialize the base model
base_svm = SVC()

# 3. Set up the Grid Search (cv=5 means 5-fold cross-validation)
grid_search = GridSearchCV(estimator=base_svm, param_grid=param_grid, cv=5, scoring='accuracy')

# 4. Run the search on your training data
grid_search.fit(X_train_scaled, y_train)

# 5. Output the best combination of hyperparameters found
print("Best Hyperparameters:", grid_search.best_params_)

# 6. Use the best model to make predictions
best_model = grid_search.best_estimator_
predictions = best_model.predict(X_test_scaled)
```

7.7 The smarter grid (pro version)

The grid above works but wastes time: it tests degree and coef0 for kernels that ignore them. GridSearchCV accepts a **list of grids**, one per kernel, so each kernel is only searched over the parameters it actually uses. We also use a Pipeline, so scaling happens *inside* each cross-validation fold (no data leakage):

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import GridSearchCV, train_test_split
from sklearn.svm import SVC

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=42)

pipe = Pipeline([('scaler', StandardScaler()), ('svm', SVC())])

param_grid = [
    {'svm__kernel': ['linear'], 'svm__C': [0.01, 0.1, 1, 10, 100]},
    {'svm__kernel': ['rbf'],    'svm__C': [0.1, 1, 10, 100],
                                'svm__gamma': ['scale', 0.001, 0.01, 0.1, 1]},
    {'svm__kernel': ['poly'],   'svm__C': [0.1, 1, 10],
                                'svm__gamma': ['scale'],
                                'svm__degree': [2, 3, 4],
                                'svm__coef0': [0.0, 1.0]},
]
```

```
grid = GridSearchCV(pipe, param_grid, cv=5, scoring='accuracy', n_jobs=-1)
grid.fit(X_train, y_train)          # raw (unscaled) data goes in; the pipeline scales it
print(grid.best_params_, grid.best_score_)
print("Test accuracy:", grid.score(X_test, y_test))   # touch the test set ONCE
```

7.8 Random search variant

```
from scipy.stats import loguniform
from sklearn.model_selection import RandomizedSearchCV

dist = {'svm__C': loguniform(1e-2, 1e3), 'svm__gamma': loguniform(1e-4, 1e1)}
rs = RandomizedSearchCV(Pipeline([('scaler', StandardScaler()), ('svm', SVC(kernel='rbf'))]),
                        dist, n_iter=40, cv=5, scoring='accuracy', n_jobs=-1, random_state=0)
rs.fit(X_train, y_train)
print(rs.best_params_)
```

NOTE · Reading the results

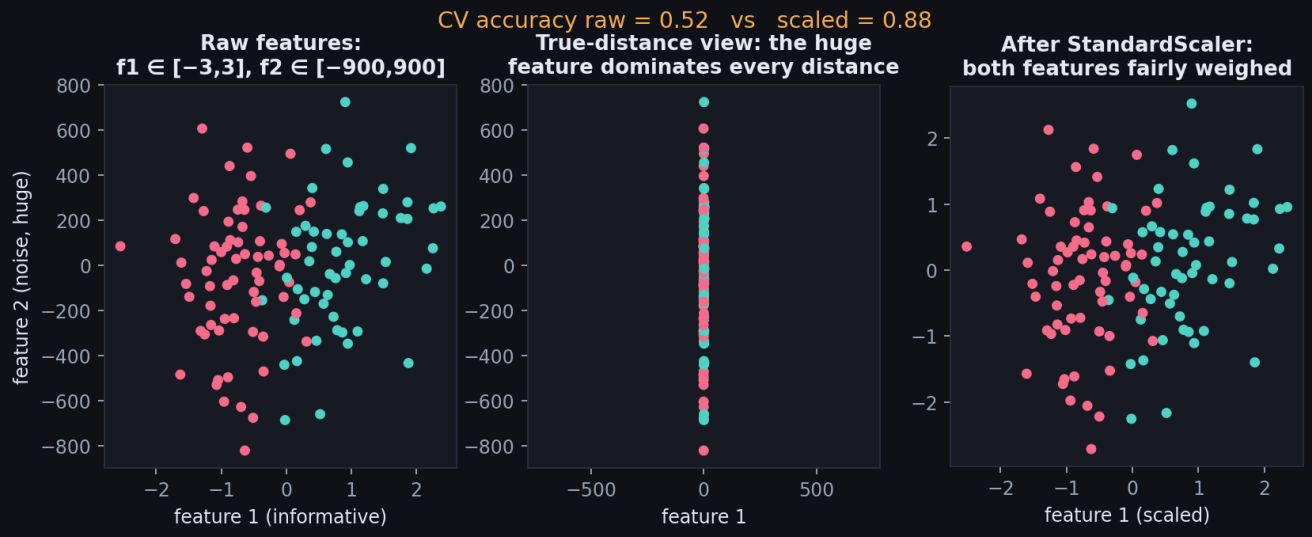
grid.best_params_ = winning hyperparameters; grid.best_score_ = mean cross-validated score of the winner (an estimate, slightly optimistic); grid.cv_results_ = a full table of every combination (turn it into a pandas DataFrame to build heat-maps like Part 6).

Part 8 · Best Practices

8.1 Feature scaling is non-negotiable

Because SVMs maximise geometric distance (margin) between points, they are highly sensitive to the magnitude of features. If one feature ranges from 0 to 1 and another from 0 to 1000, the larger feature dominates every distance calculation and effectively **blinds the model to the smaller feature**. Always scale so every feature has mean 0 and variance 1.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)   # Transform, do NOT fit the test set
```



An informative feature (small range) and a noise feature (huge range). Without scaling, distances are dominated by the noise feature and accuracy collapses.

WATCH OUT · Data leakage

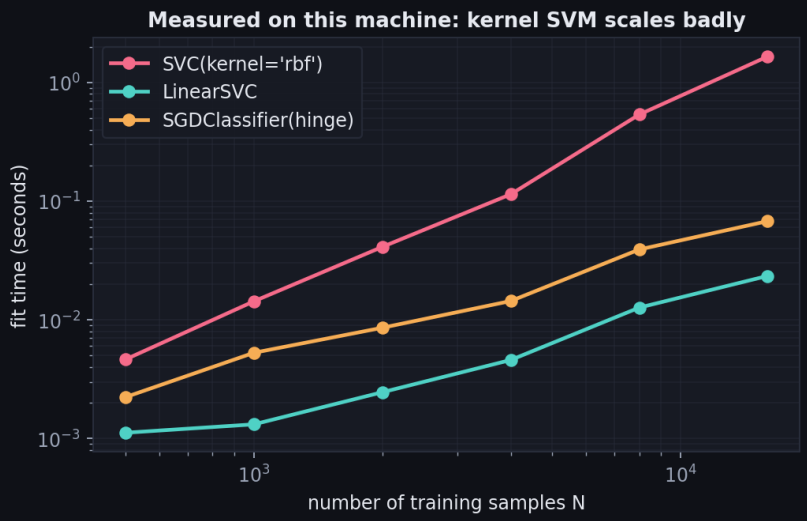
Fit the scaler on the training data only, then apply (transform) it to the test data. Fitting on the test set leaks information about the test set into training and inflates your score. A Pipeline (Part 7.7) makes this automatic.

8.2 Coarse-to-fine grid search

Covered in Part 6.4 (with a figure). Summary: search wide with large logarithmic jumps first, then zoom in with small linear steps around the best region.

8.3 Bypass standard SVC for massive datasets

The training time complexity of a standard non-linear SVM is roughly **$O(N^2)$ to $O(N^3)$**, where N is the number of samples. With over 100,000 rows, SVC(kernel='rbf') can take hours or days.



Measured fit times (log-log). Doubling the data multiplies the kernel-SVM time by roughly 3-4×; the linear methods grow almost proportionally.

- For a linear boundary on a massive dataset, use **LinearSVC()** instead of SVC(kernel='linear'). It is built on the LIBLINEAR library and is dramatically faster.
- Alternatively, use **SGDClassifier(loss='hinge')**, which approximates a linear SVM with Stochastic Gradient Descent and scales effortlessly to millions of rows.

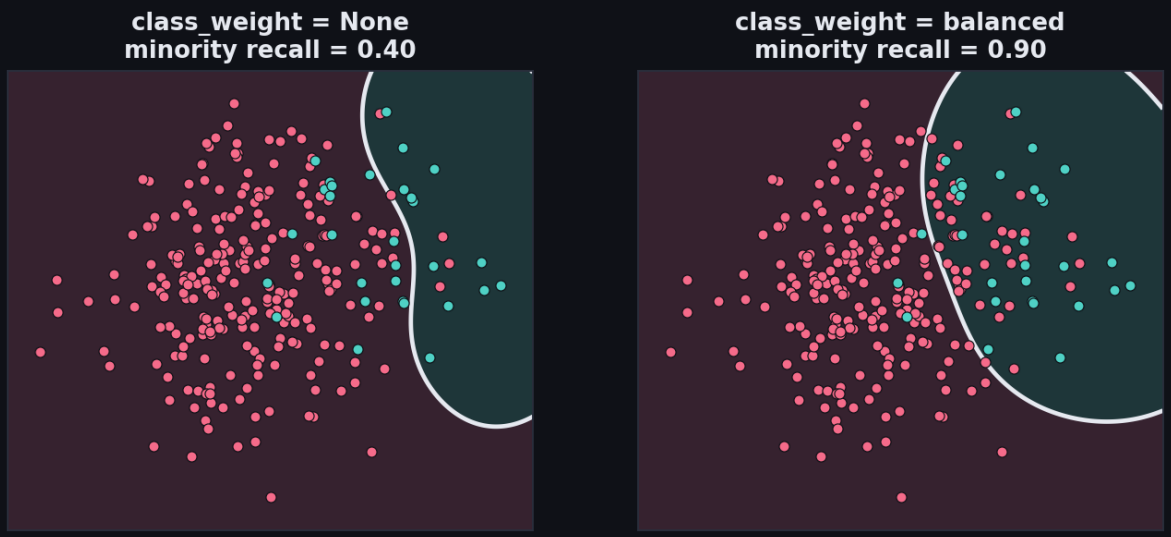
```
from sklearn.svm import LinearSVC
from sklearn.linear_model import SGDClassifier

fast_linear = LinearSVC(C=1.0)                  # LIBLINEAR
sgd_svm = SGDClassifier(loss='hinge', random_state=0)   # linear SVM via SGD
```

8.4 Neutralise class imbalance

If your dataset has 90% Class A and 10% Class B, the SVM can reach 90% accuracy by predicting Class A for everything, ignoring the minority class. Force it to pay attention by penalising errors on the minority class more heavily with **class_weight='balanced'**.

```
# Automatically adjusts weights inversely proportional to class frequencies
svm_model = SVC(kernel='rbf', class_weight='balanced')
```



Without class weights the boundary retreats towards the small class. With 'balanced' the minority class gets its region back (higher recall).

MATH · What 'balanced' does

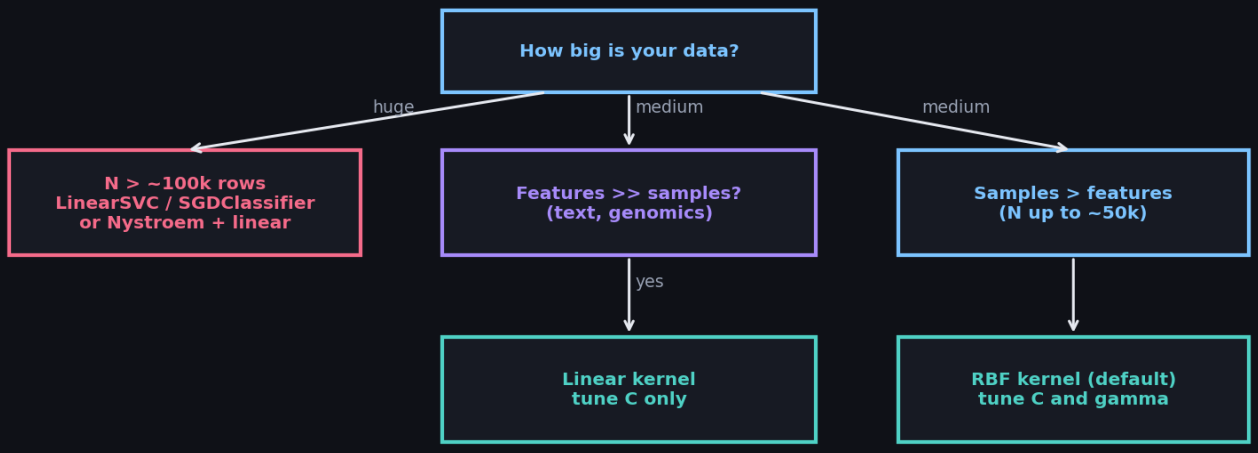
$weight_{class}$ = n_samples / (n_classes × n_samples_in_class). With 240 vs 30 samples: weight(big) = 270 / (2×240) = 0.56; weight(small) = 270 / (2×30) = 4.5. Internally each class gets its own C: C_{class} = C × weight, so an error on the small class costs 8× more.

TIP · Choose the right metric

With imbalanced data, do not judge by accuracy. Use scoring='f1', 'recall', 'balanced_accuracy' or 'roc_auc' inside GridSearchCV.

8.5 Match the kernel to the feature-to-sample ratio

- **Features > Samples** (e.g. 10,000 features, 100 samples): use a **linear** kernel. The data is already in such a high-dimensional space that mapping it higher with RBF causes immediate overfitting.
- **Samples > Features** (e.g. 50 features, 10,000 samples): use the **rbf** kernel. The non-linear mapping helps find complex relationships in dense data.



A simple decision guide for choosing an SVM variant.

8.6 Enable probability estimates only when required

By default an SVM outputs only the hard class label. To get soft probabilities (e.g. 'the model is 85% confident this is Class 1') you must set **SVC(probability=True)**. This uses **Platt scaling** (an internal logistic regression trained on top of the SVM scores, using internal cross-validation), which significantly slows down training and cross-validation. Leave it False unless you strictly need probability scores.

```
clf = SVC(kernel='rbf', probability=True)   # slower training (5-fold internal CV)
clf.fit(X_train_scaled, y_train)
proba = clf.predict_proba(X_test_scaled)    # e.g. [[0.15, 0.85], ...]
scores = clf.decision_function(X_test_scaled)  # fast raw scores, no probability=True needed
```

TIP · Cheaper alternative

If you only need a *ranking* or a threshold, use decision_function (signed distance to the boundary) which is free. Note that predict_proba can occasionally disagree with predict because they come from different internal computations.

Part 9 · Pro Tips (Community Wisdom)

These tricks are commonly shared among practitioners. Each is explained with the reason it works.

9.1 The support-vector 'health check'

Check **model.n_support_** after training. An SVM is meant to define its boundary with a *sparse* subset of the data. If the number of support vectors is close to your total number of training samples, the model is not finding a generalised boundary: it is essentially memorising the dataset (severe overfitting). Decrease gamma or lower C.

```
model = SVC(kernel='rbf', C=1, gamma=1).fit(X_train_scaled, y_train)
n_sv = model.n_support_.sum()                    # support vectors per class -> total
print(f"{n_sv} of {len(X_train_scaled)} points are support vectors "
      f"({n_sv / len(X_train_scaled):.0%})")
```

Refer back to the right panel of the validation curve in Part 6.2: as gamma grows, the ratio climbs to ~100%.

> **NOTE · Nuance**
>
> A high ratio can also be legitimate: very noisy but honest data, or a very small C (wide street swallows many points). Treat the ratio as a *warning light*, and confirm with the train-versus-test gap.

9.2 The memory-cache hack for speed

With a non-linear kernel (like RBF), the algorithm computes a massive pairwise distance/kernel matrix. By default scikit-learn allocates a small **200 MB** cache for it. If the dataset is large, the SVM constantly clears and recalculates these values, crippling speed. Increase **cache_size** (in MB) to use your RAM:

```
model = SVC(kernel='rbf', cache_size=2000)   # 2 GB kernel cache
```

9.3 Kernel approximation for big data (Nystroem / RBFSampler)

Standard non-linear SVMs scale quadratically or cubically with the number of samples, making them unusable beyond ~50,000 rows. If you need the flexibility of RBF but the speed of a linear classifier, use **Nystroem** or **RBFSampler**. They explicitly approximate the feature map φ of an RBF kernel (building a moderate number of new features), so you pipe the data through the approximator first and then into a lightning-fast LinearSVC or SGDClassifier.

```
from sklearn.kernel_approximation import Nystroem
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC

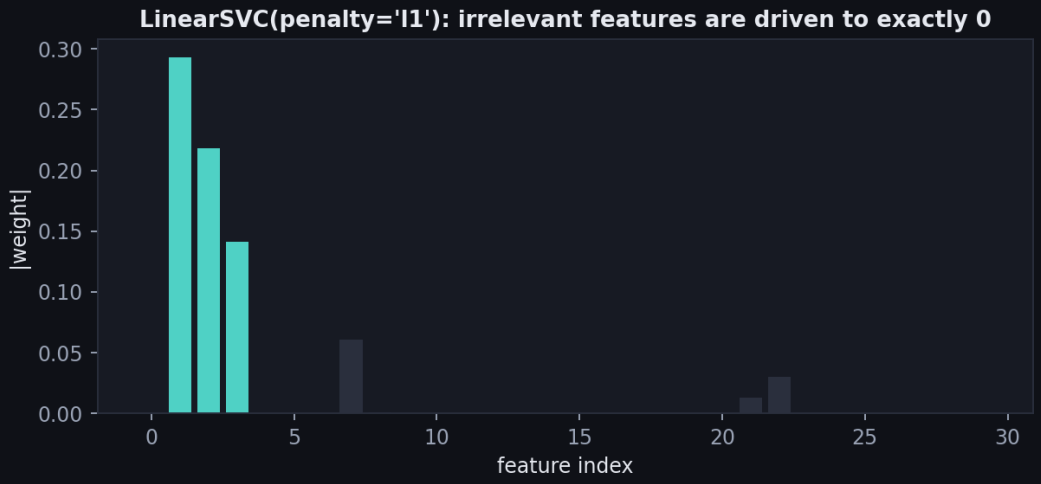
approx_svm = make_pipeline(
    StandardScaler(),
    Nystroem(kernel='rbf', gamma=0.1, n_components=300, random_state=0),  # approximate RBF map
    LinearSVC(C=1.0))                                                     # fast linear SVM
approx_svm.fit(X_train, y_train)
```

TIP · Trade-off

n_components sets the accuracy/speed balance: more components means a closer approximation of the true RBF kernel but slower training. Start around 100-500 and increase until the validation score stops improving. gamma is still tuned, exactly as for a real RBF SVM.

9.4 The L1 feature-selection pipeline

Unlike Random Forests or Lasso regression, a standard RBF SVM does **not** automatically ignore irrelevant features. Every feature contributes to the Euclidean distance, so junk data directly sabotages the margin. A professional trick: first train a **LinearSVC with an L1 penalty** (penalty='l1', dual=False) which forces the weights of irrelevant features to exactly zero. Drop those features and pass only the meaningful variables into the expensive RBF SVM.



30 features, only the first 4 are informative (teal). The L1 penalty pushes most of the junk weights to exactly zero.

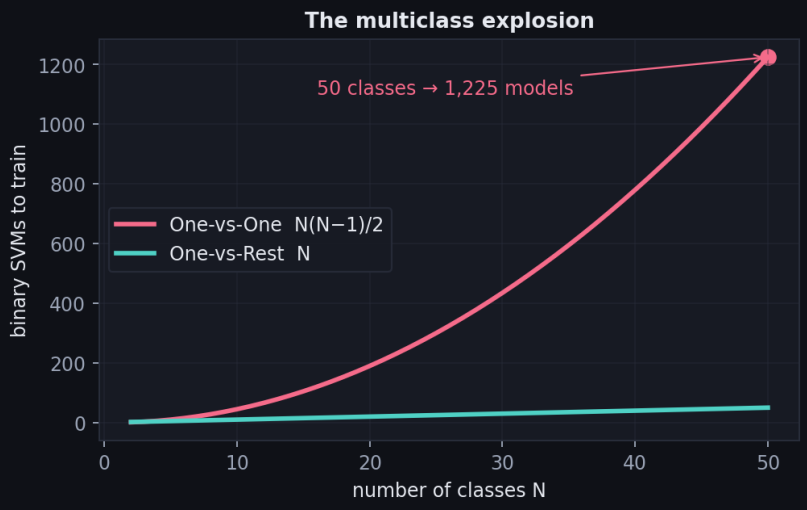
```
from sklearn.feature_selection import SelectFromModel
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC, SVC

pipe = Pipeline([
    ('scale', StandardScaler()),
    ('select', SelectFromModel(LinearSVC(penalty='l1', dual=False, C=0.05))),  # keeps non-zero weights
    ('svm', SVC(kernel='rbf', C=10, gamma='scale')),
])
pipe.fit(X_train, y_train)
print("Kept features:", pipe.named_steps['select'].get_support().sum())
```

Here the L1-penalised LinearSVC acts as a filter (its C also controls how aggressive the selection is: smaller C removes more features).

9.5 Beware the multiclass explosion (OvO vs OvR)

SVMs are fundamentally binary classifiers (Class A vs. Class B). For multiclass problems scikit-learn's SVC uses a **One-vs-One (OvO)** strategy: it trains one SVM for every pair of classes, N(N−1)/2 models. For 50 classes that is 1,225 separate SVMs, which can stall your machine. **One-vs-Rest (OvR)** trains only N models (each class against all others).



The number of binary SVMs grows quadratically with OvO but only linearly with OvR.

WATCH OUT · Important correction to a popular tip

Community advice often says 'set decision_function_shape="ovr" to train only N models'. This is **not accurate** for SVC: internally SVC (libsvm) *always* trains OvO pairs; the decision_function_shape='ovr' option only *reshapes the output scores* into N columns (and is already the default). To actually train N models, wrap it with OneVsRestClassifier, or use LinearSVC, which is OvR by default.

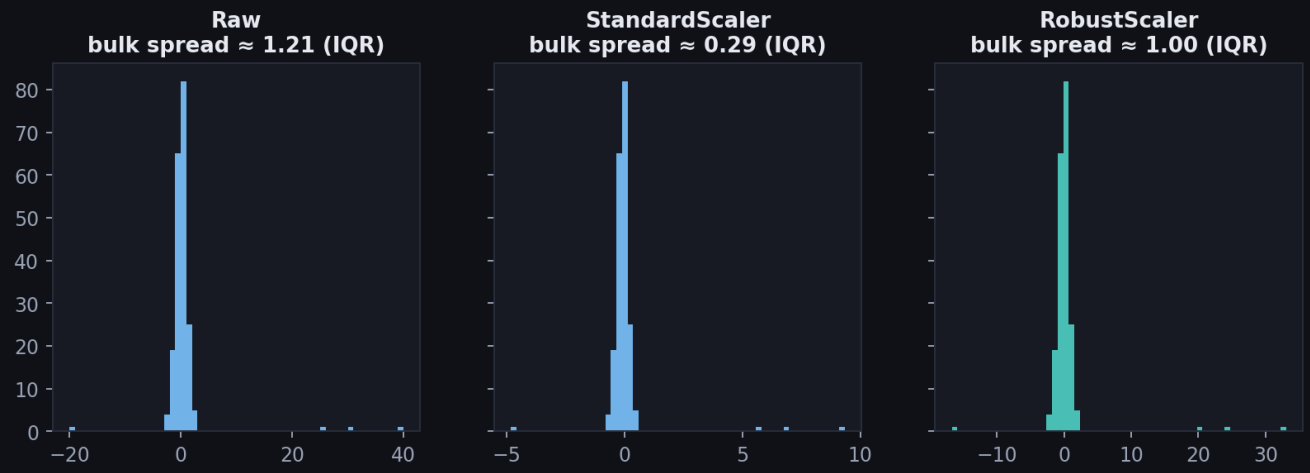
```
from sklearn.multiclass import OneVsRestClassifier
from sklearn.svm import SVC, LinearSVC

ovr_rbf = OneVsRestClassifier(SVC(kernel='rbf'))   # really trains N models
ovr_lin = LinearSVC()                              # OvR by default
```

Note that OvO is not always worse: each OvO model trains on only the two classes' data, so each is small and fast. The explosion mainly hurts with very many classes and large data.

9.6 Feature scaling with RobustScaler

Standardising is mandatory, but StandardScaler uses the mean and standard deviation, which are heavily distorted by extreme outliers. This in turn skews the SVM's distance calculations. If your data is noisy, swap it for **RobustScaler**, which uses the median and the Interquartile Range (IQR, the spread of the middle 50% of the data), making the distances immune to extreme rogue points.



200 normal values plus 4 extreme outliers. StandardScaler squashes the normal bulk into a tiny range; RobustScaler keeps it well spread.

```
from sklearn.preprocessing import RobustScaler
scaler = RobustScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

RobustScaler: x' = (x − median) / IQR IQR = Q3 − Q1 (75th minus 25th percentile)

Part 10 · Cheat Sheet, Troubleshooting and Practice

10.1 Complete end-to-end example

Copy, paste and run. It uses a built-in dataset so no downloads are needed.

```
import numpy as np
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
from sklearn.metrics import classification_report

X, y = load_breast_cancer(return_X_y=True)
X_tr, X_te, y_tr, y_te = train_test_split(X, y, test_size=0.2, stratify=y, random_state=0)

pipe = Pipeline([('scale', StandardScaler()), ('svm', SVC(class_weight='balanced'))])

# Step 1: coarse search
coarse = {'svm__C': [0.1, 10, 1000], 'svm__gamma': [0.0001, 0.01, 1]}
g1 = GridSearchCV(pipe, coarse, cv=5, n_jobs=-1).fit(X_tr, y_tr)
C0, G0 = g1.best_params_['svm__C'], g1.best_params_['svm__gamma']
print("Coarse best:", g1.best_params_, round(g1.best_score_, 4))

# Step 2: fine search around the winner
fine = {'svm__C': [C0 / 2, C0, C0 * 2], 'svm__gamma': [G0 / 2, G0, G0 * 2]}
g2 = GridSearchCV(pipe, fine, cv=5, n_jobs=-1).fit(X_tr, y_tr)
print("Fine best:  ", g2.best_params_, round(g2.best_score_, 4))

# Step 3: final exam on the untouched test set
best = g2.best_estimator_
print(classification_report(y_te, best.predict(X_te)))
print("Support vectors:", best.named_steps['svm'].n_support_.sum(), "of", len(X_tr))
```

> **NOTE · Edge-of-grid rule**
>
> In the run above the coarse winner may sit on the *edge* of your grid (for example the largest C you offered). That is a signal that the true optimum may lie beyond it, so extend the grid in that direction instead of only zooming in.

10.2 Master cheat sheet

Knob	Increase it →	Decrease it →	Start with
C	Narrower margin, stricter, more overfit risk, fewer SVs	Wider margin, smoother, underfit risk, more SVs	1, then log-scan 0.01…100
gamma	Tighter 'islands', overfit	Smoother, almost linear, underfit	'scale', then 0.001…10 (log)
degree	More curvy, overfit + slow	Simpler curve	2 or 3
coef0	More weight on lower-order terms	Pure high-order terms (r=0)	0 (or 1 for poly)
kernel	—	—	linear → rbf → poly
class_weight	—	—	'balanced' if classes are uneven
cache_size	Faster on big data (more RAM)	Slower	1000-2000 MB for big data

10.3 Troubleshooting table

Problem	Likely cause	Fix
Training takes forever	N too large for kernel SVM; small cache; probability=True; huge grid	LinearSVC / SGDClassifier / Nystroem; cache_size; drop probability; coarse-to-fine or RandomizedSearchCV
Train ~100%, test poor	Overfitting: gamma or C too high	Lower gamma then C; check n_support_
Everything predicted as majority class	Class imbalance	class_weight='balanced'; use F1/recall as scoring
Terrible accuracy from the start	Features not scaled	StandardScaler (or RobustScaler) inside a Pipeline
Poly kernel is slow or unstable	High degree; unscaled features	degree 2-3, scale features, coef0=1
Many classes, very slow	OvO explosion	OneVsRestClassifier, LinearSVC
Test score much lower than best_score_	Data leakage in preprocessing or over-tuning	Pipeline; nested cross-validation
No improvement from tuning	Irrelevant/noisy features	L1 feature selection, better features

10.4 Golden rules checklist

- ☐ Split first, scale second (fit scaler on train only) or use a Pipeline.
- ☐ Start with a linear kernel baseline, then RBF.
- ☐ Tune C and gamma jointly on log scales with cross-validation; coarse first, fine second.
- ☐ Check n_support_ and the train-versus-test gap.
- ☐ Imbalanced data → class_weight and a suitable metric.
- ☐ Large data (> ~50-100k rows) → linear or approximated kernels.
- ☐ Turn on probability=True only if you need probabilities.
- ☐ Touch the test set once, at the very end.

10.5 Glossary

Term	Meaning
Hyperparameter	A setting chosen before training (C, kernel, gamma, degree, coef0).
Bias / Variance	Bias: error from a model that is too simple (underfit). Variance: error from a model that is too sensitive to training data (overfit).
Hinge loss	max(0, 1 − y·f(x)); the penalty a point gets for being inside the margin or wrong.
Slack variable ξ	How far a point violates the margin.
Support vector	A training point on or inside the margin; it alone determines the boundary.
Kernel trick	Computing dot products in a high-dimensional space without ever leaving the original space.
Cross-validation	Repeatedly training on k−1 folds and validating on the remaining one to estimate generalisation.
Platt scaling	Fitting a sigmoid to SVM scores to obtain probabilities.
OvO / OvR	Multiclass strategies: one classifier per pair of classes, or per class against all others.
IQR	Interquartile range, Q3 − Q1; a robust measure of spread.

10.6 Practice exercises

- **Exercise 1.** Generate make_moons(300, noise=0.3). Train SVC(kernel='rbf') for C ∈ {0.01, 1, 1000} and plot the boundaries. Which overfits? Confirm with n_support_.
- **Exercise 2.** Keep C = 1 and vary gamma ∈ {0.01, 0.1, 1, 10, 100}. Plot train vs. test accuracy against gamma (a validation curve).
- **Exercise 3.** Multiply one feature by 1000 and compare cross-validated accuracy with and without StandardScaler.
- **Exercise 4.** Build a 90/10 imbalanced dataset (make_classification with weights=[0.9, 0.1]) and compare recall with and without class_weight='balanced'.
- **Exercise 5.** On 20,000 samples, time SVC(kernel='rbf') versus LinearSVC versus Nystroem + LinearSVC. Compare accuracy and speed.
- **Exercise 6.** Compute by hand: with gamma = 2 and distance 0.5, what is the RBF kernel value? (Answer: exp(−0.5) ≈ 0.607.)

10.7 Where to go next

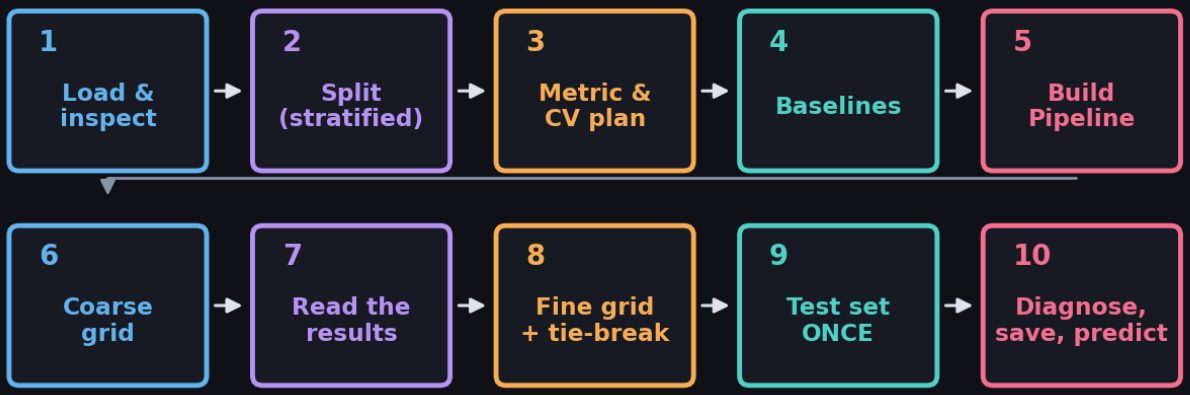
- **SVR** (Support Vector Regression): the same ideas applied to predicting numbers, with an extra epsilon tube parameter.
- **NuSVC**: an alternative parameterisation where nu (0-1) bounds the fraction of margin errors and support vectors.
- **Custom and precomputed kernels**: kernel='precomputed' lets you supply your own similarity matrix (strings, graphs, etc.).
- **Nested cross-validation**: an unbiased way to report the performance of a tuned model.

TIP · You made it

You now understand every SVM hyperparameter: what it does mathematically, how it looks visually, how to set it in code, and how to tune it like a professional. The best way to make it stick is to run the exercises above on your own data.

Part 11 · Train an SVM Like a Pro: Start to Finish

Parts 1 to 10 taught every knob one at a time. A real project needs something different: a **repeatable recipe** that takes a raw table of numbers and ends with a trustworthy, saved model. This part is that recipe. We follow ten steps in a fixed order, with real code and the **real output it produced** (scikit-learn 1.8, Python 3.12). The dataset is the Breast Cancer Wisconsin data that ships with scikit-learn: 569 patients, 30 measurements each, labelled benign or malignant. Nothing to download.



The ten steps. Steps 1-5 prepare, steps 6-8 tune, steps 9-10 judge and ship.

> **NOTE · Two rules that run through every step**
>
> **Rule 1.** The test set is used exactly once, at step 9. Every decision before that is made with cross-validation on the training set.
>
> **Rule 2.** Every learned transformation (scaling, feature selection) lives *inside* the Pipeline, so it is re-fitted on the training part of each fold and never sees validation data.

11.1 Setup and a first look at the data

Start every project the same way: imports, one random seed, and a hard look at the data. The seed (`RANDOM_STATE = 42`) makes the split, the folds and every number below reproducible.

```
import time
import numpy as np
import pandas as pd
import joblib
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, f1_score)
from sklearn.model_selection import (GridSearchCV, StratifiedKFold,
                                     cross_val_score, train_test_split)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
RANDOM_STATE = 42
```

Now load the data and ask four questions before touching any model: how big is it, is anything missing, how balanced are the classes, and how different are the feature scales?

```
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target           # y: 0 = malignant, 1 = benign
print("Shape:", X.shape)
print("Missing values:", int(X.isna().sum().sum()))
print("Class balance:\n", y.value_counts(normalize=True).round(3))
print("Feature range: min of maxes = %.3g, max of maxes = %.3g"
      % (X.max().min(), X.max().max()))
```

OUTPUT

```
Shape: (569, 30)
Missing values: 0
Class balance:
 target
1    0.627
0    0.373
Name: proportion, dtype: float64
Feature range: min of maxes = 0.0298, max of maxes = 4.25e+03
```

- **Size.** 569 rows and 30 features is about 19 samples per feature. That is not the extreme features-greater-than-samples case, so both linear and RBF are candidates (Part 8.5).
- **Missing values.** None. SVC cannot handle NaN; if you have some, add a SimpleImputer as the first pipeline step.
- **Balance.** 62.7% benign vs 37.3% malignant: mildly imbalanced, so plain accuracy can flatter a weak model (Part 8.4).
- **Scale.** The biggest value of the smallest feature is 0.03, of the largest feature 4,250: a factor of about 140,000. Unscaled, one feature would dominate every distance (Part 8.1).

WATCH OUT · Which class is the 'positive' class?

In this dataset **0 = malignant** and **1 = benign**. Metrics such as F1 treat label 1 (benign) as the positive class by default. Always check the label encoding before choosing a metric. Section 11.11 shows how to optimise for malignant recall instead.

11.2 Split first, then choose the metric and the CV plan

`test_size=0.2` keeps 80% of the rows for building the model. `stratify=y` keeps the class ratio identical in both parts, which matters with imbalance. From this line on the test set goes into a vault: nothing you learn from it may influence a decision.

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

OUTPUT

```
Train: (455, 30)  Test: (114, 30)
```

Next, decide *how you will judge* a model before you see any results. Two objects do that: a cross-validation splitter and a set of metrics.

```
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {"accuracy": "accuracy", "f1": "f1", "roc_auc": "roc_auc"}
```

Setting	What it does	Why this choice
`StratifiedKFold(5, shuffle=True)`	Cuts the training data into 5 folds, each with the same class ratio, after shuffling.	Rows are often sorted by class or time; shuffling avoids lopsided folds. Use 10 folds for small data, 5 for speed.
`accuracy`	Share of correct predictions.	Easy to read but misleading under imbalance.
`f1`	Harmonic mean of precision and recall of class 1.	Punishes both false alarms and misses.
`roc_auc`	How well the scores rank the classes (uses decision_function automatically).	Threshold-free, so it is not affected by where you cut.
`refit='f1'`	After the search, retrain on the whole training set using the F1 winner.	With several metrics you must name the one that picks the winner.

TIP · One metric decides, the others watch

Track several metrics, but choose **one** to pick the winner, and choose it before you look at any results. Switching metric after seeing scores is a quiet form of cheating.

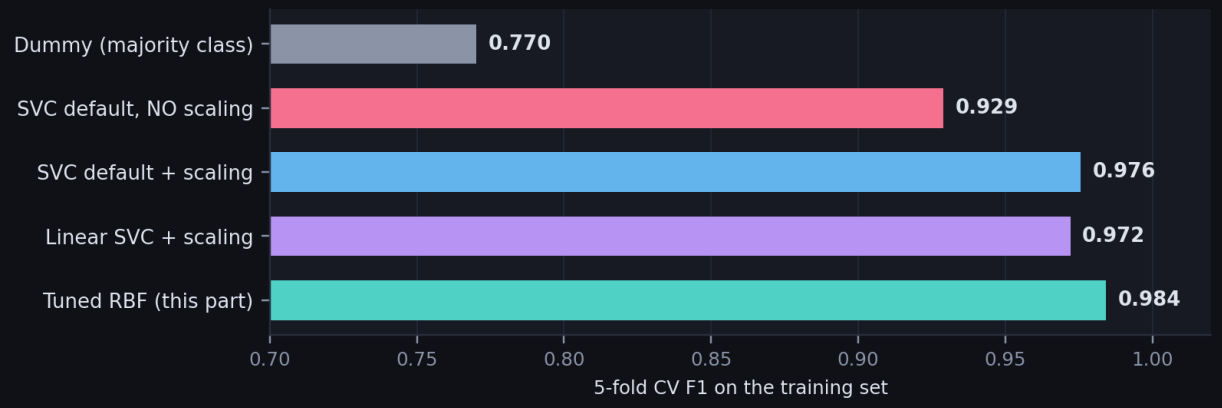
11.3 Baselines: know what you must beat

Never tune before you know what 'good' looks like. Four quick models give you the floor and a realistic ceiling. Each one is scored with the same 5-fold CV on the training set.

```
def cv_f1(model):
    return cross_val_score(model, X_train, y_train, cv=cv, scoring="f1").mean()
baselines = {
    "Dummy (majority class)":       DummyClassifier(strategy="most_frequent"),
    "SVC default, NO scaling":      SVC(),
    "SVC default + scaling":        Pipeline([("scale", StandardScaler()), ("svm", SVC())]),
    "Linear SVC + scaling":         Pipeline([("scale", StandardScaler()), ("svm", SVC(kernel="linear"))]),
}
base_scores = {name: cv_f1(m) for name, m in baselines.items()}
for k, v in base_scores.items():
    print(f"{k:28s} CV F1 = {v:.4f}")
```

OUTPUT

```
Dummy (majority class)       CV F1 = 0.7703
SVC default, NO scaling      CV F1 = 0.9289
SVC default + scaling        CV F1 = 0.9756
Linear SVC + scaling         CV F1 = 0.9721
```



Baseline CV F1 scores, with the tuned model added at the bottom for comparison.

- **Dummy = 0.770.** Answering 'benign' for everybody already scores 0.77 F1. Any real model must clear this floor.

- **Scaling = +4.7 points.** Going from 0.929 to 0.976 needed no tuning at all. It is the single biggest gain in this whole part (Part 8.1).
- **Linear is almost as good.** 0.972 versus 0.976 says the classes are nearly linearly separable, so tuning has little headroom. Expect small gains, and be suspicious of large ones.

11.4 Build the Pipeline

```
pipe = Pipeline([("scale", StandardScaler()),
                 ("svm", SVC(class_weight="balanced", random_state=RANDOM_STATE))])
```

A Pipeline chains steps into one estimator. Calling *fit* runs the scaler's fit-transform on the training rows and then trains the SVC; calling *predict* scales with the stored mean and standard deviation and then predicts. Inside GridSearchCV the whole chain is re-fitted for every fold, so a validation fold never influences the scaler. That is how leakage is prevented (Part 8.1).

The step names (`scale`, `svm`) matter: a parameter of the SVC is addressed as `svm__C`, `svm__gamma`, `svm__kernel` (name, **double** underscore, parameter). We also set `class_weight="balanced"` because of the 63/37 imbalance (Part 8.4).

> **TIP · List the valid parameter names**
> Not sure how to spell one? Run `pipe.get_params().keys()`. A typo in the grid raises *Invalid parameter*, which is much better than silently searching nothing.

11.5 Stage 1: coarse grid search

The coarse stage asks one question: **which region is worth exploring?** Use wide logarithmic steps and one grid per kernel, so each kernel is searched only over the parameters it actually uses (Part 7.7).

```
coarse_grid = [
    {"svm__kernel": ["linear"],
     "svm__C": [0.001, 0.01, 0.1, 1, 10, 100]},
    {"svm__kernel": ["rbf"],
     "svm__C": [0.01, 0.1, 1, 10, 100, 1000],
     "svm__gamma": [1e-4, 1e-3, 1e-2, 1e-1, 1]},
    {"svm__kernel": ["poly"],
     "svm__C": [0.1, 1, 10],
     "svm__degree": [2, 3],
     "svm__coef0": [0.0, 1.0],
     "svm__gamma": ["scale"]},
]
coarse = GridSearchCV(pipe, coarse_grid, cv=cv, scoring=scoring, refit="f1",
                      n_jobs=-1, return_train_score=True)
t0 = time.time()
coarse.fit(X_train, y_train)
print("Coarse search took %.1f s" % (time.time() - t0))
print("\nCOARSE best:", coarse.best_params_, " CV F1 = %.4f" % coarse.best_score_)
n_fits = len(coarse.cv_results_["params"]) * cv.get_n_splits()
print("Coarse candidates:", len(coarse.cv_results_["params"]), " fits:", n_fits)
```

Argument	Meaning
`cv=cv`	The stratified 5-fold splitter from step 3, so every candidate is judged on identical folds.
`scoring=scoring, refit='f1'`	Compute three metrics per candidate; pick the winner by F1.
`n_jobs=-1`	Use all CPU cores. The candidates are independent, so this is a near-linear speed-up.
`return_train_score=True`	Also record training scores. Needed to see the train/validation gap, the overfitting alarm (Part 6.2).

Fit budget: linear has 6 candidates, rbf has 6 × 5 = 30, poly has 3 × 2 × 2 × 1 = 12. That is 48 candidates × 5 folds = **240 fits**, finished in 7.2 seconds here.

OUTPUT

```
COARSE best: {'svm__C': 100, 'svm__gamma': 0.0001, 'svm__kernel': 'rbf'}  CV F1 = 0.9843
Coarse candidates: 48  fits: 240
```

WATCH OUT · Grids grow multiplicatively

Fits = candidates × folds. Adding just one more value to each of four parameters multiplies the grid by 2.4 or more. Keep the budget in mind and use the coarse-to-fine approach instead of one giant grid (Part 6.3).

11.6 Read the results like a pro

Do not stop at `best_params_`. The full leaderboard lives in `cv_results_`; turn it into a DataFrame and look at the top rows, including the training score and the fold-to-fold standard deviation.

```
res = pd.DataFrame(coarse.cv_results_)
top = (res.sort_values("rank_test_f1")
          [["param_svm__kernel", "param_svm__C", "param_svm__gamma",
            "mean_test_f1", "std_test_f1", "mean_train_f1"]].head(8))
print(top.to_string(index=False))
```

OUTPUT

param_svm__kernel	param_svm__C	param_svm__gamma	mean_test_f1	std_test_f1	mean_train_f1
rbf	100.00	0.0001	0.984287	0.006524	0.987796
rbf	10.00	0.01	0.982546	0.010956	0.991700
poly	1.00	scale	0.982516	0.007849	0.990376
linear	0.10	NaN	0.980838	0.008392	0.988623
linear	0.01	NaN	0.980777	0.008597	0.985632
rbf	1.00	0.01	0.980777	0.006608	0.986013
rbf	100.00	0.001	0.980777	0.006608	0.990810
rbf	10.00	0.001	0.980748	0.003417	0.986920

Three things a professional reads from this table:

- **A plateau, not a podium.** Rank 1 scores 0.9843 ± 0.0065, but ranks 2 to 8 are within 0.004 of it and their standard deviations are 0.003 to 0.011. Differences smaller than the standard deviation are noise.
- **Edge of the grid.** The winner uses gamma = 0.0001, the smallest value offered. The true optimum may lie further out, so the fine stage must extend past it (the edge-of-grid rule from Part 10.1).
- **Train/validation gap.** Rank 1 has 0.9878 train vs 0.9843 validation: a gap of 0.0035, healthy. Rank 2 (C = 10, gamma = 0.01) has 0.9917 vs 0.9825, a gap of 0.0092, so it leans more towards overfitting even though its score is similar.

TIP · Build the heat-map yourself

Filter one kernel and pivot: `rbf.pivot(index="param_svm__gamma", columns="param_svm__C", values="mean_test_f1")`. Plot it with imshow and you get the C × gamma heat-map of Part 6.1 for your own data (left panel of the figure in 11.7).

11.7 Stage 2: fine search and deliberate tie-breaking

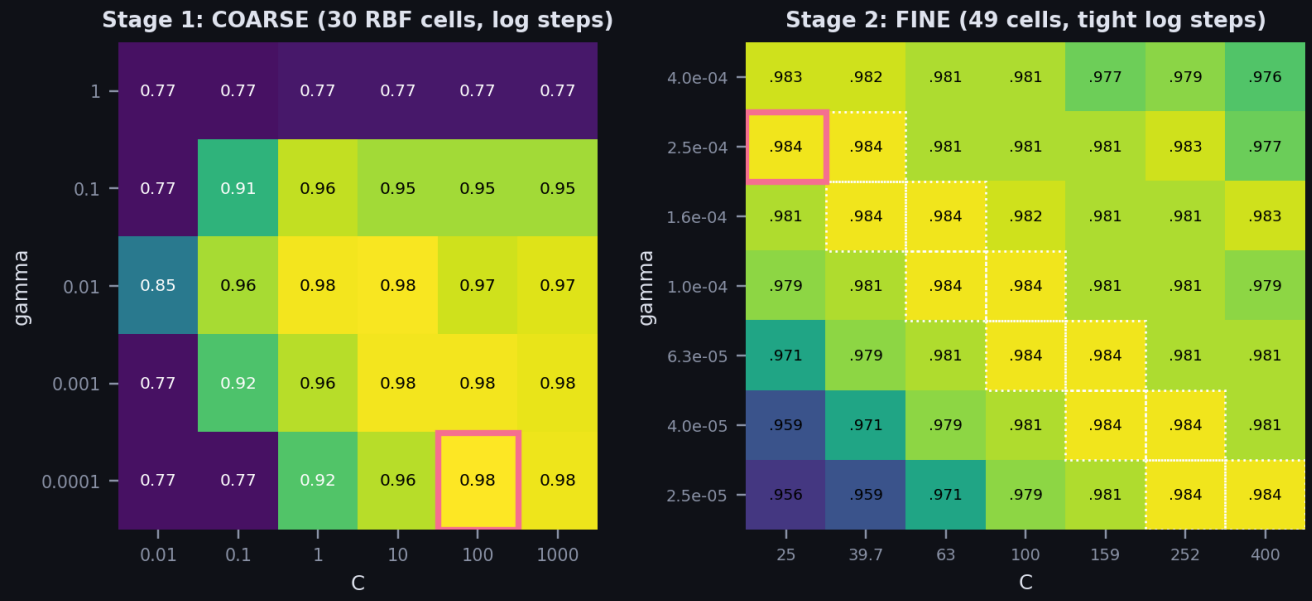
Now zoom into the promising region. We build the fine grid *from the coarse winner* instead of typing numbers: seven values from a quarter to four times the winning C and gamma. The steps are geometric, not linear, because C and gamma act multiplicatively (Part 2.3). The range also reaches below gamma = 0.0001, which handles the edge-of-grid warning from 11.6.

```
best = coarse.best_params_
if best["svm__kernel"] == "rbf":
    C0, g0 = best["svm__C"], best["svm__gamma"]
    fine_grid = {"svm__kernel": ["rbf"],
                 "svm__C": np.round(np.geomspace(C0 / 4, C0 * 4, 7), 6).tolist(),
                 "svm__gamma": np.round(np.geomspace(g0 / 4, g0 * 4, 7), 8).tolist()}
    fine = GridSearchCV(pipe, fine_grid, cv=cv, scoring=scoring, refit="f1",
                        n_jobs=-1, return_train_score=True)
    t0 = time.time()
    fine.fit(X_train, y_train)
    print("Fine search took %.1f s" % (time.time() - t0))
    print("\nFINE best:", fine.best_params_, " CV F1 = %.4f" % fine.best_score_)
    final_search = fine
else:
    final_search = coarse
```

OUTPUT

```
FINE best: {'svm__C': 25.0, 'svm__gamma': 0.00025198, 'svm__kernel': 'rbf'}  CV F1 = 0.9843
```

49 candidates × 5 folds = 245 fits in 5.5 seconds. The best CV F1 is **0.9843, exactly the same** as in stage 1. That is a result, not a failure: stage 1 already found the plateau, and stage 2 confirmed it and revealed its shape.



Left: coarse RBF grid (rose box = winner). Right: fine grid; dotted white cells are the 12 candidates tied at the best score, rose box = the model we keep. The ties form the diagonal ridge of Part 6.1.

Pro move: break ties on purpose

Twelve of the 49 candidates score exactly 0.9843. By default GridSearchCV returns the first one in grid order, which is arbitrary. They lie on the diagonal ridge from Part 6.1: **large C with small gamma** gives the same fit as **small C with large gamma**. A deliberate rule is better than luck. The **one-standard-error rule**: among all candidates whose score is within one standard error of the best, take the **simplest** model. Here simplest means the smallest C (strongest regularization, Part 2.1), then the smallest gamma. GridSearchCV supports this directly: `refit` may be a function that receives `cv_results_` and returns the index of the chosen row.

```
def simplest_within_one_se(cv_results):
    """Among candidates within 1 standard error of the best F1,
    pick the smoothest model: smallest C, then smallest gamma."""
    mean = np.asarray(cv_results["mean_test_f1"])
    se = np.asarray(cv_results["std_test_f1"]) / np.sqrt(cv.get_n_splits())
    ok = np.flatnonzero(mean >= mean.max() - se[mean.argmax()])
    C = np.asarray(cv_results["param_svm__C"], dtype=float)[ok]
    g = np.asarray(cv_results["param_svm__gamma"], dtype=float)[ok]
    return int(ok[np.lexsort((g, C))[0]])          # sort by C, then gamma
if best["svm__kernel"] == "rbf":
    simple = GridSearchCV(pipe, fine_grid, cv=cv, scoring=scoring,
                          refit=simplest_within_one_se, n_jobs=-1,
                          return_train_score=True)
    simple.fit(X_train, y_train)
    print("Simplest-within-1-SE:", simple.best_params_)
    n_tied = int((np.asarray(simple.cv_results_["mean_test_f1"]) >= fine.best_score_ - 1e-12).sum())
    print("Candidates tied with the best score:", n_tied, "of", len(simple.cv_results_["params"]))
    final_search = simple
final_model = final_search.best_estimator_
```

OUTPUT

```
Simplest-within-1-SE: {'svm__C': 25.0, 'svm__gamma': 0.00025198, 'svm__kernel': 'rbf'}
Candidates tied with the best score: 12 of 49
```

NOTE · A callable refit has no best_score_

When `refit` is a function, scikit-learn sets `best_params_` and `best_estimator_` but **not** `best_score_`. Read scores from `cv_results_` using `best_index_`, as step 10 does below.

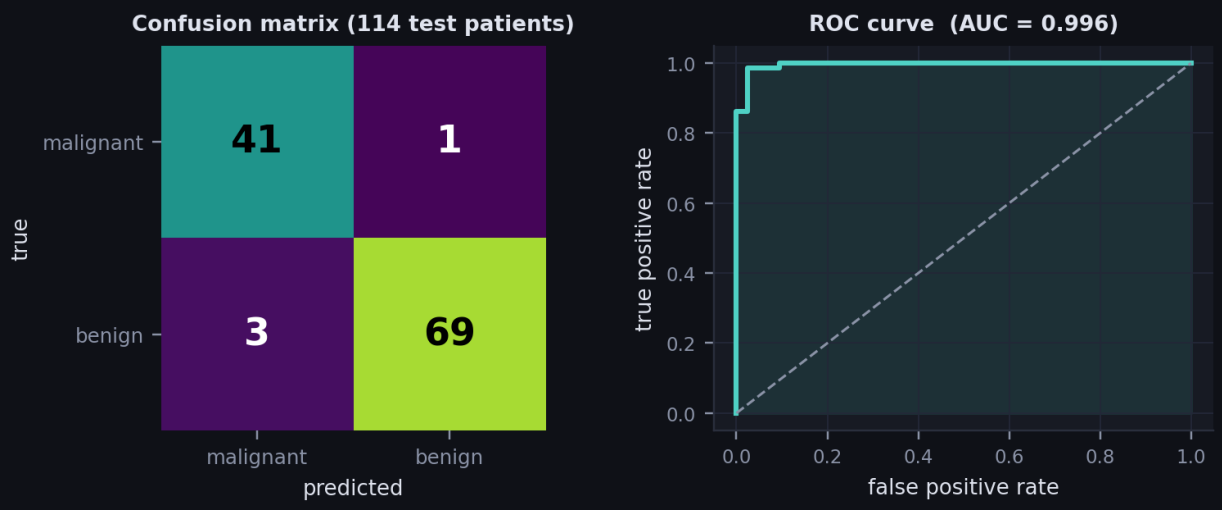
11.8 The final exam: evaluate on the test set

This is the only time the vault opens. Predict on the untouched 114 patients and report several views of performance, not a single number. For ROC-AUC we use `decision_function`, which needs no `probability=True` and therefore no slow Platt scaling (Part 8.6).

```
y_pred = final_model.predict(X_test)
y_score = final_model.decision_function(X_test)
print("\nTEST results")
print(classification_report(y_test, y_pred, target_names=["malignant", "benign"], digits=3))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print("ROC-AUC = %.4f" % roc_auc_score(y_test, y_score))
```

OUTPUT

```
TEST results
              precision    recall  f1-score   support
   malignant      0.932     0.976     0.953        42
      benign      0.986     0.958     0.972        72
    accuracy                         0.965       114
   macro avg      0.959     0.967     0.963       114
weighted avg      0.966     0.965     0.965       114
Confusion matrix:
 [[41  1]
 [ 3 69]]
ROC-AUC = 0.9957
```



Confusion matrix and ROC curve on the 114 held-out patients.

- **Reading the matrix.** Rows are the truth, columns the prediction: 41 malignant found, **1 malignant missed** (called benign), 3 benign flagged as malignant, 69 benign correct.
- **Cross-validation vs test.** CV F1 was 0.984 and the test F1 for benign is 0.972. On 114 patients one case moves a score by about one point, so this gap is within noise. The result is consistent, not disappointing.
- **Which error is worse?** In screening, the single missed malignancy costs more than three false alarms. Section 11.11 shows how to tune for that.

> **WATCH OUT · One look only**
>
> If you go back and change hyperparameters because of this test score, the test set has silently become a validation set and no longer gives an honest estimate. Accept the number, or get fresh data, or use nested cross-validation (next section).

11.9 Diagnose, verify, save and predict

Health check (Part 9.1 and Part 6.2)

```
svm = final_model.named_steps["svm"]
n_sv = svm.n_support_.sum()
print("Support vectors: %d of %d (%.0f%%)" % (n_sv, len(X_train), 100 * n_sv / len(X_train)))
r = pd.DataFrame(final_search.cv_results_).iloc[final_search.best_index_]
print("Chosen cell: train F1 = %.4f  vs  CV F1 = %.4f" % (r["mean_train_f1"], r["mean_test_f1"]))
```

OUTPUT

```
Support vectors: 102 of 455 (22%)
Chosen cell: train F1 = 0.9865  vs  CV F1 = 0.9843
Reloaded model agrees on test set: True
```

Only 22% of the training points are support vectors, far from the near-100% that signals memorisation. Train F1 (0.9865) and CV F1 (0.9843) are almost equal, so there is no overfitting: in the language of Part 6.2, a **good fit**. Stop tuning.

An honest estimate of the whole procedure: nested CV

`best_score_` is slightly optimistic, because the same folds both picked the winner and scored it. **Nested cross-validation** runs the entire search inside every outer fold, so each outer test fold is judged by a model whose tuning never saw it. We use a smaller 4 × 4 grid to keep it fast.

```
inner = StratifiedKFold(5, shuffle=True, random_state=1)
outer = StratifiedKFold(5, shuffle=True, random_state=2)
small_grid = {"svm__C": [0.1, 1, 10, 100], "svm__gamma": [1e-3, 1e-2, 1e-1, 1]}
nested_search = GridSearchCV(pipe, small_grid, cv=inner, scoring="f1", n_jobs=-1)
nested = cross_val_score(nested_search, X, y, cv=outer, scoring="f1")
print("\nNested CV F1: %.4f +/- %.4f" % (nested.mean(), nested.std()))
```

OUTPUT

```
Nested CV F1: 0.9820 +/- 0.0093
```

A nested score of 0.982 ± 0.009 against 0.984 from the search shows that the tuning procedure is trustworthy on this dataset. On small or noisy data with big grids the gap can be much larger, which is exactly why you check.

Save the model and predict on new data

```
joblib.dump(final_model, "svm_breast_cancer_v1.joblib")
loaded = joblib.load("svm_breast_cancer_v1.joblib")
print("Reloaded model agrees on test set:", bool((loaded.predict(X_test) == y_pred).all()))
new_patient = X_test.iloc[[0]]                    # raw, unscaled numbers: the pipeline scales them
label = int(loaded.predict(new_patient)[0])
print("Prediction:", data.target_names[label],
      "| decision score = %.2f" % loaded.decision_function(new_patient)[0],
      "| truth:", data.target_names[int(y_test.iloc[0])])
```

OUTPUT

```
Prediction: malignant | decision score = -4.18 | truth: malignant
```

The new patient is passed as **raw, unscaled numbers**: the saved Pipeline contains the fitted scaler and applies it automatically. The decision score is negative, so the class is 0 (malignant). A score of −4.18 lies far beyond the margin edge at −1 (Part 1.4), a confident call.

WATCH OUT · Saved models are tied to library versions

A `.joblib` file is a pickle: load it with the **same scikit-learn version** that saved it, and record that version next to the file. Never load a model file from a source you do not trust, because unpickling can execute arbitrary code.

11.10 The whole script in one piece

Everything above, assembled. Save it as `svm_pro_pipeline.py` and run `python svm_pro_pipeline.py`; it finishes in well under a minute and prints exactly the output shown in this part. To reuse it, change only the dataset lines (step 1), the metric (step 3) and the grids (steps 6 and 7).

```
import time
import numpy as np
import pandas as pd
import joblib
from sklearn.datasets import load_breast_cancer
from sklearn.dummy import DummyClassifier
from sklearn.metrics import (classification_report, confusion_matrix,
                             roc_auc_score, f1_score)
from sklearn.model_selection import (GridSearchCV, StratifiedKFold,
                                     cross_val_score, train_test_split)
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.svm import SVC
RANDOM_STATE = 42
# ---------------------------------------------------------------------- 1. LOAD
data = load_breast_cancer(as_frame=True)
X, y = data.data, data.target            # y: 0 = malignant, 1 = benign
print("Shape:", X.shape)
print("Missing values:", int(X.isna().sum().sum()))
print("Class balance:\n", y.value_counts(normalize=True).round(3))
print("Feature range: min of maxes = %.3g, max of maxes = %.3g"
```

```
      % (X.max().min(), X.max().max()))
# ---------------------------------------------------------------------- 2. SPLIT
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y, random_state=RANDOM_STATE)
print("Train:", X_train.shape, " Test:", X_test.shape)
# ---------------------------------------------------------------------- 3. CV + METRICS
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)
scoring = {"accuracy": "accuracy", "f1": "f1", "roc_auc": "roc_auc"}
# ---------------------------------------------------------------------- 4. BASELINES
def cv_f1(model):
    return cross_val_score(model, X_train, y_train, cv=cv, scoring="f1").mean()
baselines = {
    "Dummy (majority class)":       DummyClassifier(strategy="most_frequent"),
    "SVC default, NO scaling":      SVC(),
    "SVC default + scaling":        Pipeline([("scale", StandardScaler()), ("svm", SVC())]),
    "Linear SVC + scaling":         Pipeline([("scale", StandardScaler()), ("svm", SVC(kernel="linear"))]),
}
base_scores = {name: cv_f1(m) for name, m in baselines.items()}
for k, v in base_scores.items():
    print(f"{k:28s} CV F1 = {v:.4f}")
# ---------------------------------------------------------------------- 5. PIPELINE
pipe = Pipeline([("scale", StandardScaler()),
                 ("svm", SVC(class_weight="balanced", random_state=RANDOM_STATE))])
# ---------------------------------------------------------------------- 6. COARSE SEARCH
coarse_grid = [
    {"svm__kernel": ["linear"],
     "svm__C": [0.001, 0.01, 0.1, 1, 10, 100]},
    {"svm__kernel": ["rbf"],
     "svm__C": [0.01, 0.1, 1, 10, 100, 1000],
     "svm__gamma": [1e-4, 1e-3, 1e-2, 1e-1, 1]},
    {"svm__kernel": ["poly"],
     "svm__C": [0.1, 1, 10],
     "svm__degree": [2, 3],
     "svm__coef0": [0.0, 1.0],
     "svm__gamma": ["scale"]},
]
coarse = GridSearchCV(pipe, coarse_grid, cv=cv, scoring=scoring, refit="f1",
                      n_jobs=-1, return_train_score=True)
t0 = time.time()
coarse.fit(X_train, y_train)
print("Coarse search took %.1f s" % (time.time() - t0))
print("\nCOARSE best:", coarse.best_params_, " CV F1 = %.4f" % coarse.best_score_)
n_fits = len(coarse.cv_results_["params"]) * cv.get_n_splits()
print("Coarse candidates:", len(coarse.cv_results_["params"]), " fits:", n_fits)
# ---------------------------------------------------------------------- 6b. READ THE COARSE RESULTS
res = pd.DataFrame(coarse.cv_results_)
top = (res.sort_values("rank_test_f1")
          [["param_svm__kernel", "param_svm__C", "param_svm__gamma",
            "mean_test_f1", "std_test_f1", "mean_train_f1"]].head(8))
print(top.to_string(index=False))
# ---------------------------------------------------------------------- 7. FINE SEARCH
best = coarse.best_params_
if best["svm__kernel"] == "rbf":
    C0, g0 = best["svm__C"], best["svm__gamma"]
    fine_grid = {"svm__kernel": ["rbf"],
                 "svm__C": np.round(np.geomspace(C0 / 4, C0 * 4, 7), 6).tolist(),
                 "svm__gamma": np.round(np.geomspace(g0 / 4, g0 * 4, 7), 8).tolist()}
    fine = GridSearchCV(pipe, fine_grid, cv=cv, scoring=scoring, refit="f1",
                        n_jobs=-1, return_train_score=True)
    t0 = time.time()
    fine.fit(X_train, y_train)
    print("Fine search took %.1f s" % (time.time() - t0))
    print("\nFINE best:", fine.best_params_, " CV F1 = %.4f" % fine.best_score_)
    final_search = fine
else:
    final_search = coarse
# ---------------------------------------------------------------------- 7b. BREAK TIES ON PURPOSE
def simplest_within_one_se(cv_results):
    """Among candidates within 1 standard error of the best F1,
    pick the smoothest model: smallest C, then smallest gamma."""
    mean = np.asarray(cv_results["mean_test_f1"])
    se = np.asarray(cv_results["std_test_f1"]) / np.sqrt(cv.get_n_splits())
    ok = np.flatnonzero(mean >= mean.max() - se[mean.argmax()])
    C = np.asarray(cv_results["param_svm__C"], dtype=float)[ok]
    g = np.asarray(cv_results["param_svm__gamma"], dtype=float)[ok]
    return int(ok[np.lexsort((g, C))[0]])          # sort by C, then gamma
if best["svm__kernel"] == "rbf":
```

```
    simple = GridSearchCV(pipe, fine_grid, cv=cv, scoring=scoring,
                          refit=simplest_within_one_se, n_jobs=-1,
                          return_train_score=True)
    simple.fit(X_train, y_train)
    print("Simplest-within-1-SE:", simple.best_params_)
    n_tied = int((np.asarray(simple.cv_results_["mean_test_f1"]) >= fine.best_score_ - 1e-12).sum())
    print("Candidates tied with the best score:", n_tied, "of", len(simple.cv_results_["params"]))
    final_search = simple
final_model = final_search.best_estimator_
# ---------------------------------------------------------------- 8. TEST ONCE
y_pred = final_model.predict(X_test)
y_score = final_model.decision_function(X_test)
print("\nTEST results")
print(classification_report(y_test, y_pred, target_names=["malignant", "benign"], digits=3))
print("Confusion matrix:\n", confusion_matrix(y_test, y_pred))
print("ROC-AUC = %.4f" % roc_auc_score(y_test, y_score))
# ---------------------------------------------------------------- 9. HEALTH CHECK
svm = final_model.named_steps["svm"]
n_sv = svm.n_support_.sum()
print("Support vectors: %d of %d (%.0f%%)" % (n_sv, len(X_train), 100 * n_sv / len(X_train)))
r = pd.DataFrame(final_search.cv_results_).iloc[final_search.best_index_]
print("Chosen cell: train F1 = %.4f  vs  CV F1 = %.4f" % (r["mean_train_f1"], r["mean_test_f1"]))
# ---------------------------------------------------------------- 10. SAVE
joblib.dump(final_model, "svm_breast_cancer_v1.joblib")
loaded = joblib.load("svm_breast_cancer_v1.joblib")
print("Reloaded model agrees on test set:", bool((loaded.predict(X_test) == y_pred).all()))
# ---------------------------------------------------------------- 11. PREDICT ON NEW DATA
new_patient = X_test.iloc[[0]]                    # raw, unscaled numbers: the pipeline scales them
label = int(loaded.predict(new_patient)[0])
print("Prediction:", data.target_names[label],
      "| decision score = %.2f" % loaded.decision_function(new_patient)[0],
      "| truth:", data.target_names[int(y_test.iloc[0])])
# ---------------------------------------------------------------- BONUS: nested CV
inner = StratifiedKFold(5, shuffle=True, random_state=1)
outer = StratifiedKFold(5, shuffle=True, random_state=2)
small_grid = {"svm__C": [0.1, 1, 10, 100], "svm__gamma": [1e-3, 1e-2, 1e-1, 1]}
nested_search = GridSearchCV(pipe, small_grid, cv=inner, scoring="f1", n_jobs=-1)
nested = cross_val_score(nested_search, X, y, cv=outer, scoring="f1")
print("\nNested CV F1: %.4f +/- %.4f" % (nested.mean(), nested.std()))
```

11.11 Pro upgrades, checklist and practice

Adapting the recipe to other situations

Situation	Change to make
The rare class matters most (screening, fraud)	Optimise its recall: `make_scorer(recall_score, pos_label=0)` as `scoring` (tested below).
More than two classes	`scoring="f1_macro"`; consider OneVsRestClassifier or LinearSVC (Part 9.5).
Big grid or many rows	`HalvingGridSearchCV` or `RandomizedSearchCV` (Part 7.8); LinearSVC or Nystroem beyond ~100k rows (Part 9.3).
Extreme outliers	Swap StandardScaler for RobustScaler inside the Pipeline (Part 9.6).
Many junk features	Add a SelectFromModel(LinearSVC(penalty='l1')) step to the Pipeline (Part 9.4).
Rows are time-ordered	Use `TimeSeriesSplit` instead of StratifiedKFold; never shuffle the future into the past.
Several rows per patient or user	Use `GroupKFold` so one person never appears in both training and validation.

Two of these, with real results from this dataset. First, tuning for malignant recall instead of benign F1 (label 0 is the class we do not want to miss):

```
from sklearn.metrics import make_scorer, recall_score
recall_malignant = make_scorer(recall_score, pos_label=0)
gs = GridSearchCV(pipe, {"svm__C": [1, 10, 100], "svm__gamma": [1e-3, 1e-2]},
                  cv=cv, scoring=recall_malignant, n_jobs=-1).fit(X_train, y_train)
```

OUTPUT

```
{'svm__C': 10, 'svm__gamma': 0.001}   CV recall of malignant = 0.9647
```

Second, successive halving: it trains every candidate on a small slice of the data, keeps the best third, and gives the survivors more data each round.

```
from sklearn.experimental import enable_halving_search_cv  # noqa: F401
from sklearn.model_selection import HalvingGridSearchCV
hs = HalvingGridSearchCV(pipe, {"svm__C": [0.1, 1, 10, 100, 1000], "svm__gamma": [1e-4, 1e-3, 1e-2, 1e-1]},
                         cv=cv, scoring="f1", factor=3, random_state=42, n_jobs=-1).fit(X_train, y_train)
```

OUTPUT

```
{'svm__C': 10, 'svm__gamma': 0.01}   CV F1 = 0.9822   (1.7 s for 20 candidates)
```

Pre-flight checklist: before you trust the model

☐ Data inspected: shape, missing values, class balance, feature scales, label encoding.

☐ Stratified split done **before** anything is fitted; test set untouched.

☐ Scaler (and any selector) inside the Pipeline, never fitted on all the data.

☐ Metric chosen for the real-world cost of errors, not by habit.

☐ Baselines recorded: dummy, default SVC, linear.

☐ Coarse then fine search; winner not stuck on the edge of the grid.

☐ Ties broken by a rule; train/validation gap and n_support_ checked.

☐ Test set evaluated once; nested CV run if the estimate must be defended.

☐ Model saved with the library version; prediction tested on raw data.

Practice exercises

- **Exercise 7.** Re-run the whole script on *load_wine* (3 classes). Change the scoring to f1_macro. Which kernel wins, and does the fine stage improve on the coarse stage?
- **Exercise 8.** In 11.7, change the tie-break rule to 'largest C within one standard error'. Does the test score change? What does that tell you about the plateau?
- **Exercise 9.** Replace StandardScaler with RobustScaler and add a copy of one feature multiplied by 1000 with a few extreme outliers. Compare the CV F1 of both scalers.
- **Exercise 10.** Time the coarse search with n_jobs=1 and n_jobs=-1. How close to linear is the speed-up on your machine?

TIP · You can now train an SVM end to end

You know the order of the steps, why each one exists, how to read a grid search instead of trusting it, and how to defend the final number. Apply the same ten steps to your own data and you will avoid the mistakes that sink most SVM projects.